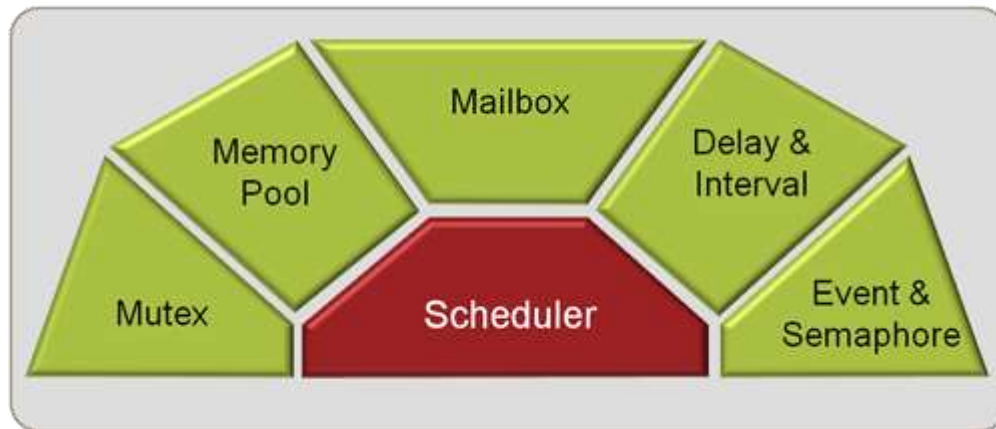


RTOS and RTx51



What To Be Covered?

- **RTOS**
- **Scheduling Introduction**
 - Review of Software Architecture
 - CPU Scheduling
 - Data Transfer & Message Queues
 - Synchronisation
 - Process Execution Characteristics
- **Scheduler Implementation in RTOS**
 - Scheduler structure
 - Scheduling Policies (Types of Schedulers)
- **Scheduling Analysis**
- **Process Interactions & Priority Inversion**
- **Multi-Processor Scheduling**
- **Scheduler Summary and Conclusion**
- **Examples of Scheduling Applications**
- **RTx51**

OS and RTOS

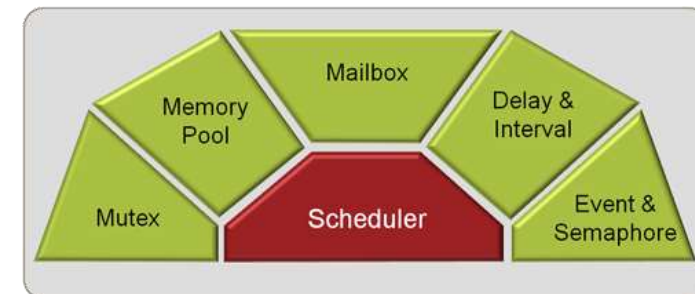
- ❑ **Operating System (OS)** allocate resources in the computing system among programs that request them. It also schedules processes to meet performance requirements and provide services beyond CPU scheduling.
- ❑ **Multitasking:** Processing codes for the different tasks as directed by the scheduler.
- ❑ **Kernel:** A program with functions for memory allocation and de-allocation, task scheduling, inter-process communication, effective management of shared memory access by using the signals, exception (error) handling signals, queues, mailboxes, pipes and sockets, I/O management, interrupts control, device drivers and device management.
- ❑ **Real Time Operating Systems (RTOS):** An OS for real-time programming and scheduling, process and memory manager, device drivers, device management and multitasking.
- ❑ RTOS has to be **scalable** in embedded systems.
- ❑ **Scalable OS** is one in which memory is optimized by having only that part of features that are needed and is associated with the final software.

What is a Kernel?

- The kernel is a piece of software
 - It has to share the CPU with all the rest of application code
- The kernel is the part of the OS that handles:
 - Process management
 - E.g. process creation, deletion, suspension
 - Process scheduling
 - Ensures that the most important code runs first
 - Allow multitasking
 - Application is broken into multiple processes
 - Inter-process communications and synchronization
 - Message queues, semaphores
 - Mutual exclusion
 - Semaphore management



The RTOS kernel contains a scheduler that runs program code as tasks. Communication between tasks is accomplished by RTOS objects such as events, semaphores, mutexes and mailboxes. Additional RTOS services include time and memory management and interrupt support.



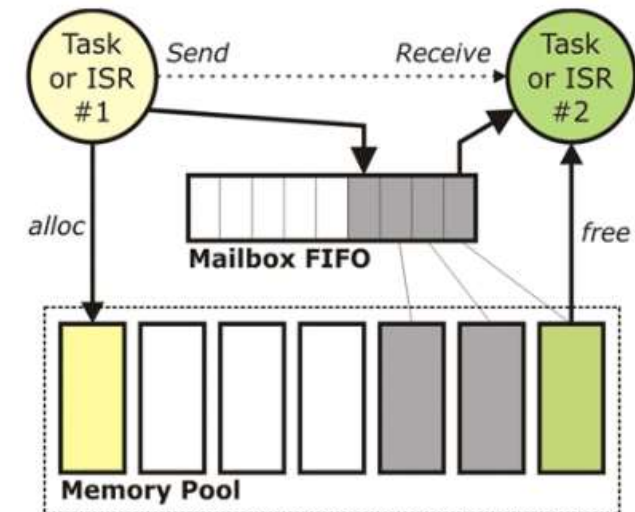
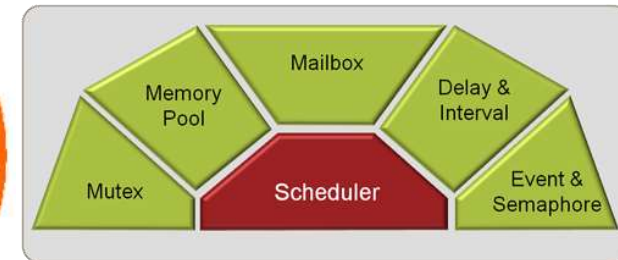
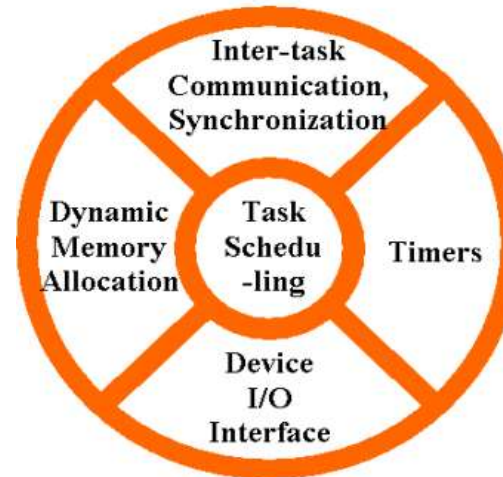
- A typical micro kernel may only have 10 to 12 functions
 - Interrupts are usually locked as we enter the kernel
 - The kernel typically gets to run anytime we return from interrupt

WHY-RTOS: BENEFITS

A RTOS separates the program functions into self-contained tasks and implements an on-demand scheduling of their execution.

An advanced RTOS, such as the Keil RTX, delivers serious benefits:

- **Task scheduling** - tasks are called when needed ensuring better program flow and event response
- **Multitasking** - task scheduling gives the illusion of executing a number of tasks simultaneously
- **Deterministic behaviour** - events and interrupts are handled within a defined time
- **Shorter ISRs** - enables more deterministic interrupt behaviour
- **Inter-task communication** - manages the sharing of data, memory, and hardware resources among multiple tasks
- **Defined stack usage** - each task is allocated a defined stack space, enabling predictable memory usage
- **System management** - allows you to focus on application development rather than resource management (housekeeping)



Tasks 1 & 2 are functions in the application. RTX provides management of the Memory Pool and Mailbox as well as the communication mechanisms used between the tasks.

```
os_mbx_declare (mailbox1, 20);

__task void task1 (void) {
    void *msg;

    os_mbx_init (mailbox1, sizeof(mailbox1));
    msg = alloc();
    // fill message content

    os_mbx_send (mailbox1, msg, 12);
}
```

```
__task void task2 (void) {
    void *msg;
    ..
    os_mbx_wait (mailbox1, &msg, 100);
    // process message content here
    free (msg);
}
```

RTX provides a simple syntax for straightforward access to all of the RTOS resources.

WHY-RTOS: SUPERLOOP DISADVANTAGES

Simple embedded systems typically use a *Super-Loop* concept where the application executes each function in a fixed order. Interrupt Service Routines (ISR) are used for time-critical program portions.

This approach is well suited for small systems but has limitations for more complex applications.

- **Disadvantages of the *Super-Loop* Concept**

- Time-critical operations must be processed within interrupts (ISR)
 - ISR functions become complex and require long execution times
 - ISR nesting may create unpredictable execution time and stack requirements
- Data exchange between *Super-Loop* and ISR is via global shared variables
 - Application programmer must ensure data
- A *Super-Loop* can be easily synchronized with the System timer, but:
 - If a system requires several different cycle times, it is hard to implement
 - Split of time-consuming functions that exceed *Super-Loop* cycle
 - Creates software overhead and application program is hard to understand
- *Super-Loop* applications become complex and therefore hard to extend
 - A simple change may have unpredictable side effects; such side effects are time consuming to analyze.

Features of Real-time Systems

- ❑ **Timing:** Real-time systems should **compute** and **deliver** correct results **within** a **specified** period of **time**.
- ❑ **Interrupt Driven:** real-time systems involve **event triggering** (mostly in terms of interrupt signals arising from the arrival of data at an input port, or the ticking of a hardware clock, or an error status alarm)
- ❑ **Low-level Programming:** coding done at assembler level as C do not have necessary instructions to handle interrupt processing. ASM and C are classified as low-level language by many programmers.
- ❑ **Specialized Hardware:** the programmers must have the knowledge of other disciplines such as electrical theory, mechanics, acoustics, physiology or optics etc. This is because the **programmer has to fully understand the functional role of the software** and its contribution to the feedback delay which governs the system response.

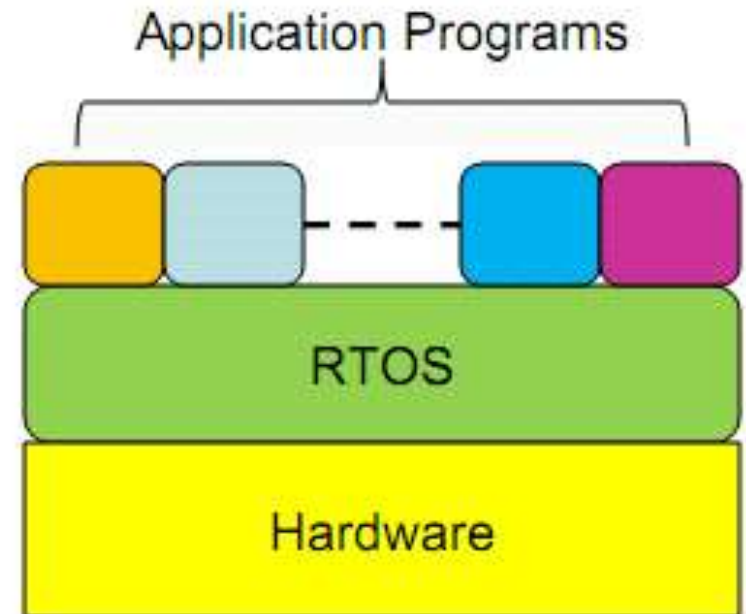
Features of Real-time Systems

- ❑ **Volatile data I/O:** these are **variables** which change their values from moment to moment. **Most commonly input channels** which operate autonomously to bring in new values for memory variables when data arrives at an input port.
- ❑ **Multi-tasking:** several processes or tasks cooperate to carry out the overall job in a real-time system. **System partitioning** is done such that manageable **components run in parallel** or concurrently.
- ❑ **Run-time Scheduling:** **sequencing of tasks** takes place at run-time and is accomplished **by the OS** through the action of the **scheduler** (a kind of 'late sequencing' i.e. sequencing decisions have been deferred).
- ❑ **Unpredictability:** unpredictable changes in the environments of real-time systems **require considerable amount of simulation and testing** as exact ordering or sequencing of all the functions which deal with these interactions are decided at the run-time by the scheduler.
- ❑ **Life-critical Code:** **Software reliability** and **systems integrity** is a big concern for life-critical systems (cost will increase).

Real-Time Operating Systems (RTOS)

❑ **Application Software:**

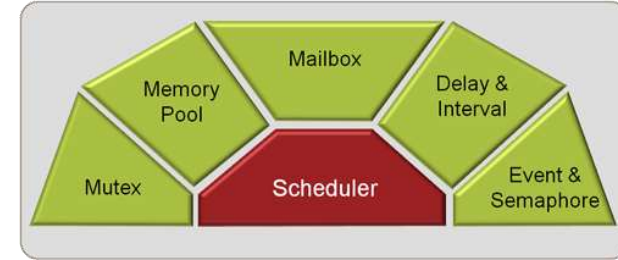
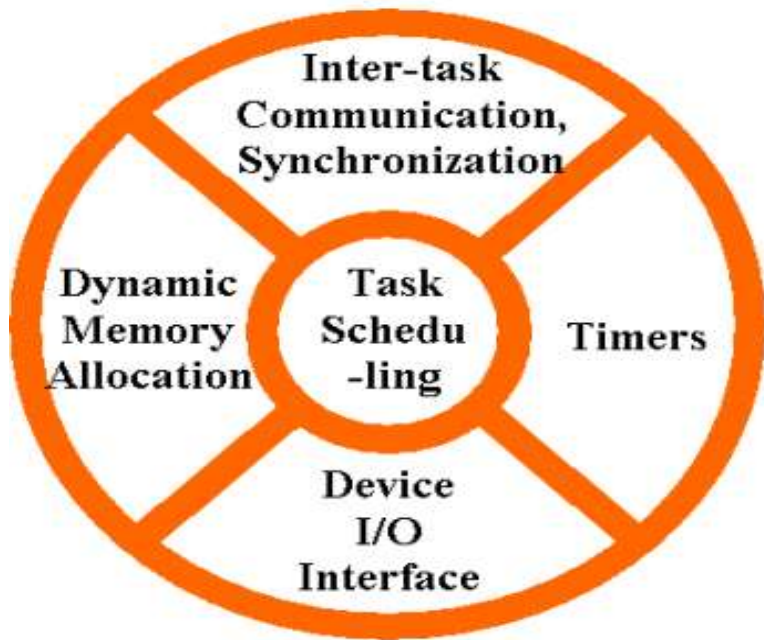
- several tasks
- decided at the design time
- tasks represent a specific device
- time constraints, synchronization and communication is important between tasks



❑ **RTOS:**

- deadline (software timer)
- scheduling (task runtime)
- resource allocation (sharing)
- intertask communication (signaling, passing information etc.)
- dynamic memory allocation

Anatomy of an RTOS



RTOS Design Goals

- Portability
 - Minimise code specific to a given architecture/compiler
- Scalability
 - Make feature-set configurable, so the same system can be configured for very small, or large, systems
- Performance
 - Low interrupt-level & task-level latency
 - Interested in maximum limit, not average
 - Low RAM use
- Rich Feature-set?
 - Ways to deal with priority-inversion
 - Deadline scheduling
 - Rich set of comms & synch primitives

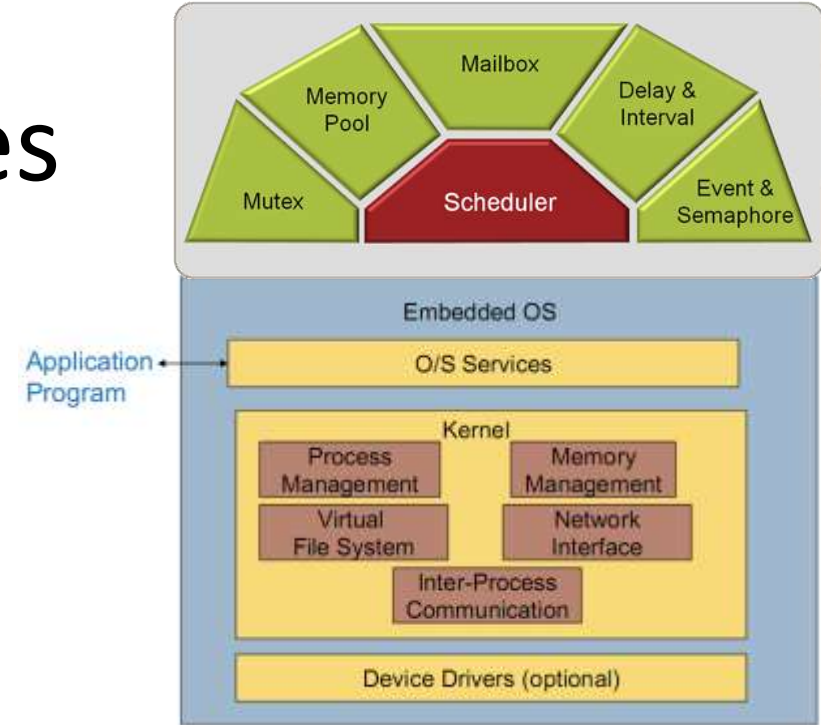
- Hardware interface
 - Interrupt handling
 - Task switching
 - Clock tick implementation
- Scheduling
- Communication & Synchronisation
- Memory Allocation
- Timers & Delay Implementation

Why RTOS and Scheduling?

- Existing as a separate code layer, RTOS makes your coding better in sense of flexibility, clarity, portability, ...
- Especially useful for applications of complexity with many tasks of timing constraints, deadlines, ...
 - When more tasks are added, RTOS/scheduler becomes important
- The applications based on RTOS/Schedulers are inherently predictable, safe and reliable
- RTOS/scheduler supports team working, since individual tasks can often be developed largely independently and then assembled into the final system.
- Obtain determined & rapid responses to external events

RTOS Features

- Main features:
 - Task scheduling
 - Priority, time-slice, fixed ordering, etc.
 - Meet real-time requirements
 - Inter-task communication
 - Task synchronization & mutual exclusion
 - Protect tasks from each other
 - Memory management
 - Scalability
 - Library of plug-ins at compile time to minimize size of RTOS
- Other features:
 - Date/time, File system, Networking, Security ...



General OS model (Linux-like)

RTOS Terminologies

- ❑ **RTOS:** Operating systems (OS) that provide facilities for satisfying real-time requirements (by **allocating resources** using certain real-time algorithms).
- ❑ **Process:** a **single execution of a program** (has its own state that includes registers and memory)
- ❑ **Threads:** Processes that share the same address space (in RTOS)
- ❑ **Multirate Systems:** certain **operations** are **executed periodically** and each operation is executed **at its own rate**.
- ❑ **Release Time:** the **time** at which the **process** becomes **ready to execute** (not necessarily the time at which it actually takes control of the CPU and starts to run)

RTOS Terminologies

- ❑ **Deadline:** specifies when a computation must be finished.
 - for **aperiodic** process => measured from the release time
 - for **periodic** process => occur at some time other than the end of the period
- ❑ **Process's Period:** the time between successive executions
- ❑ **Process's Rate:** inverse of its period
- ❑ **Scheduler:** A typical RTOS may have a kernel, a hardware abstraction layer, an executive and some device drivers. The **scheduler** is that **part of the kernel** that is **responsible for handling events**.
- ❑ **Tasks:** operations performed to complete a specific purpose
- ❑ **Task Execution Time:** The CPU time required to execute a task.

RTOS Terminologies

- ❑ **Task Period:** the time interval between tasks (a task period of 100ms => task must be started and completed every 100ms).
- ❑ **Task Deadline:** If task period = T and the completion requirement = D , where $D < T$ => such requirement is called a task deadline (default deadline is the task period).
- ❑ **Task Rate:** reciprocal of the task period.
- ❑ **Transport Lag:** time elapsed between measuring (sensing) and generating an output (actuating) of a system. The smaller the transport lag, the better the stability and accuracy of a system.

SW Design using RTOS

Software Design using RTOS

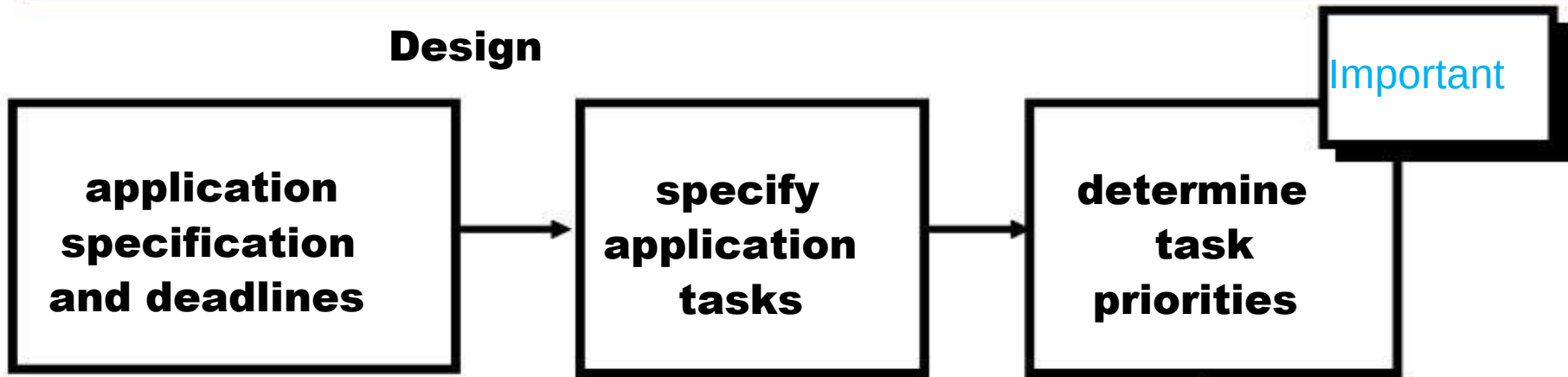
- ❑ Each Task has a distinct memory location
- ❑ Tasks are made up of one or more procedures
- ❑ Shared Memory Architecture (Graphic chips doesn't have own dedicated memory, instead shares main system RAM with CPU and other components)
- ❑ Distributed memory Architecture – UMA/ NUMA
- ❑ Scheduling done by OS - takes care of constraints

- ❑ RTOS kernel schedules transitions of task
 - Ready state -> Running State
 - Block state -> Running State (pre-emptive scheduling)
 - Tasks can enter into ready state at any time
 - At a time only one task will be in running state

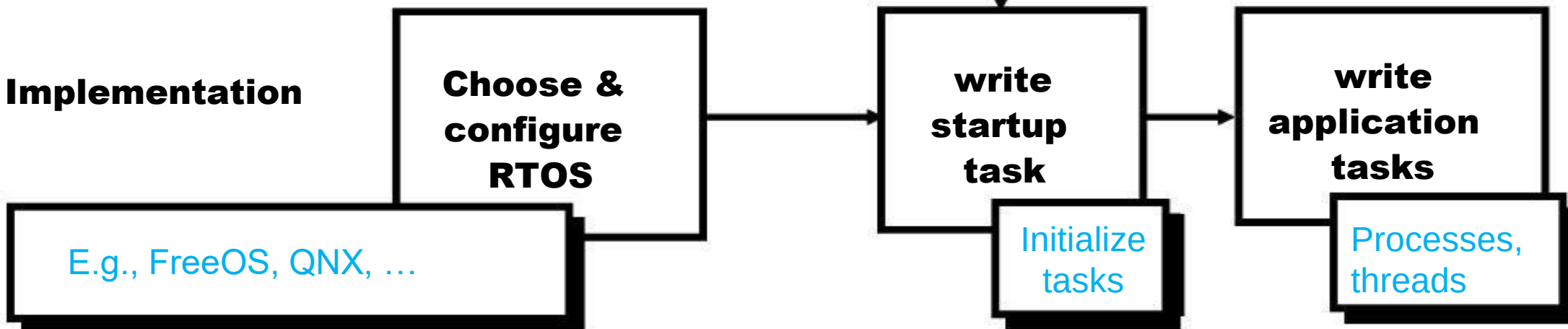
- ❑ RTOS also controls inter-process communication

Real-Time System Design: Summary

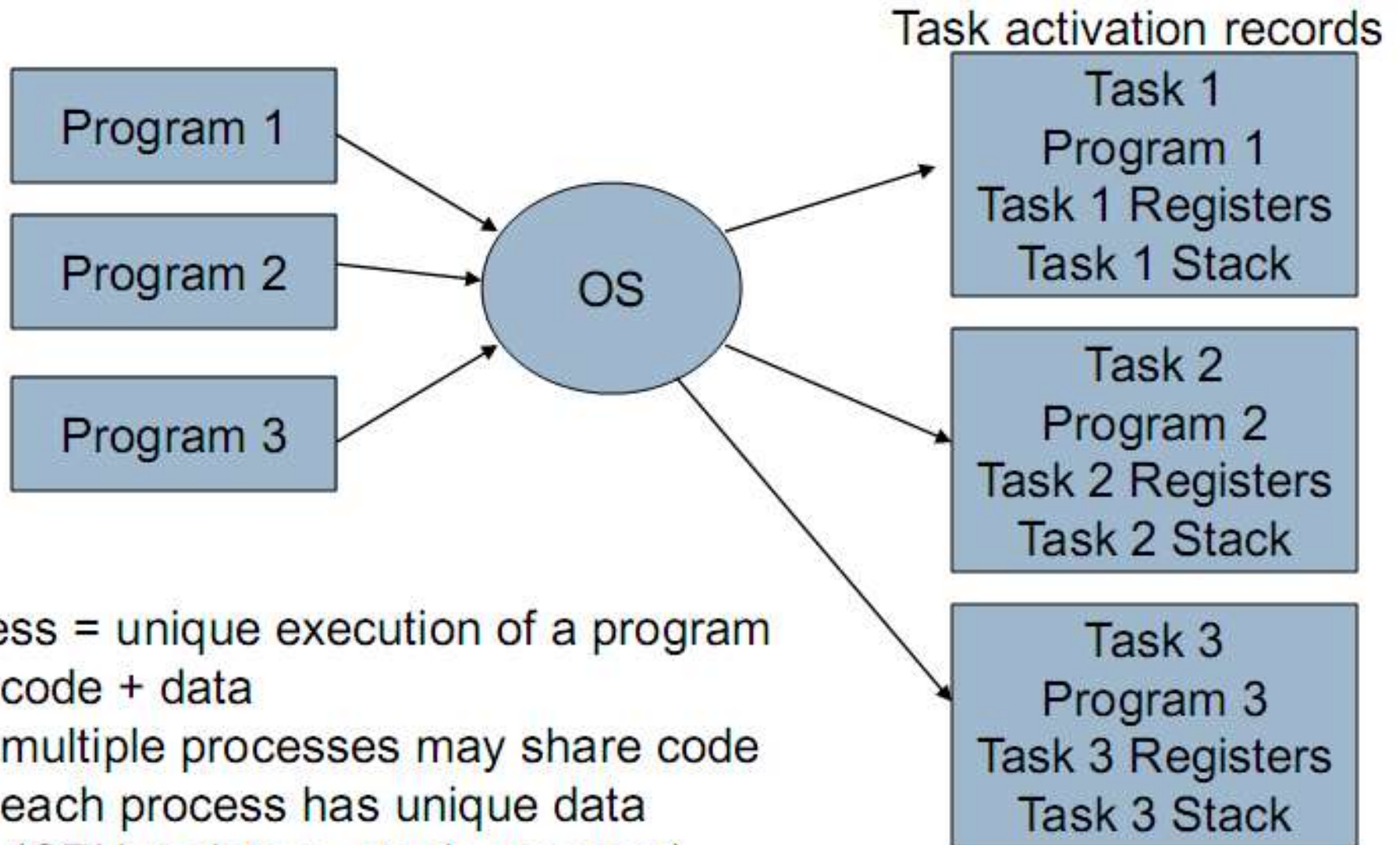
Design



Implementation



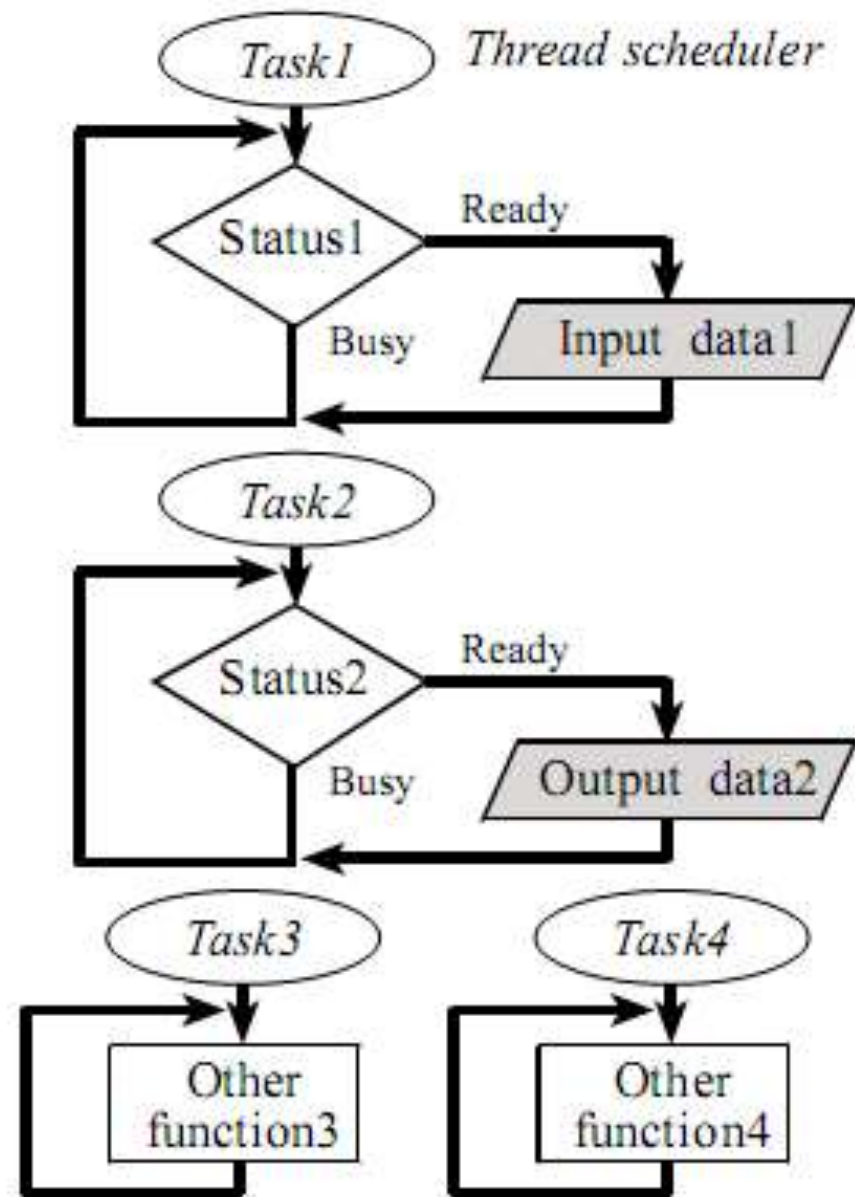
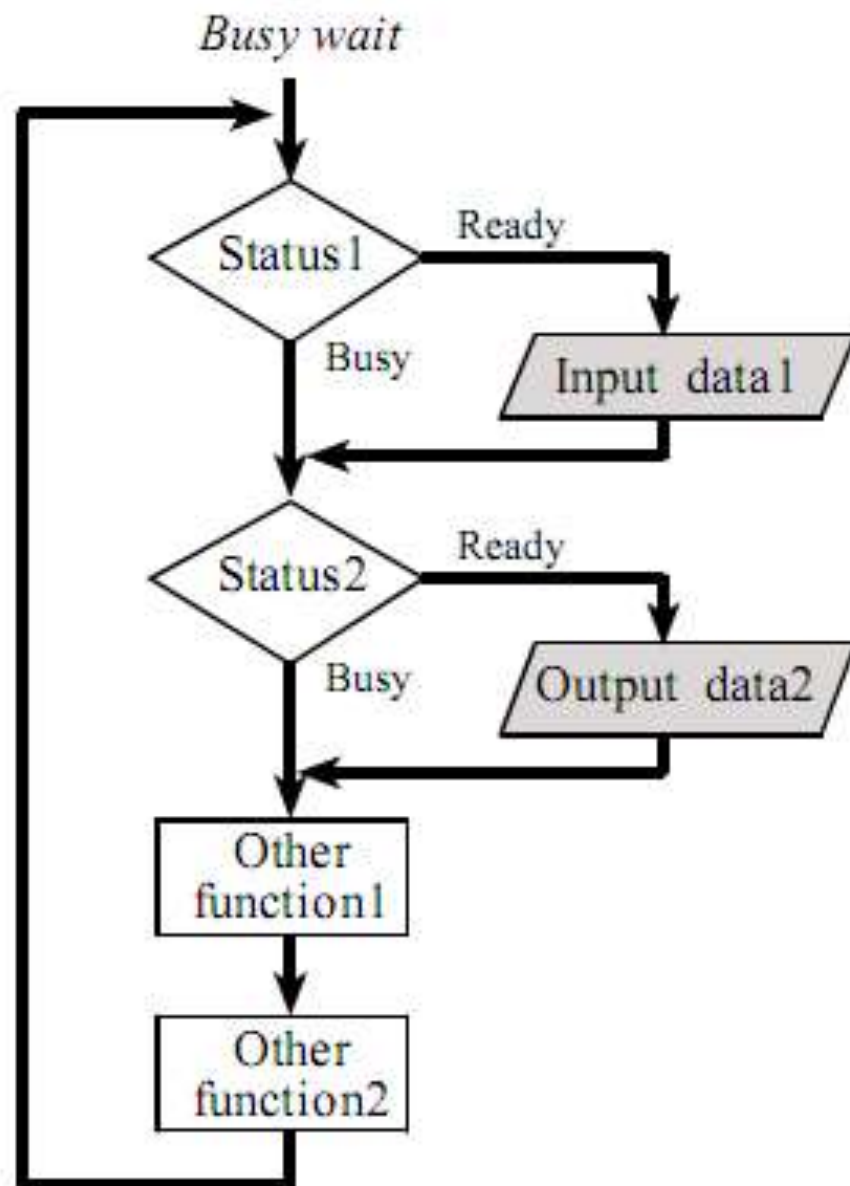
Multitasking in OS



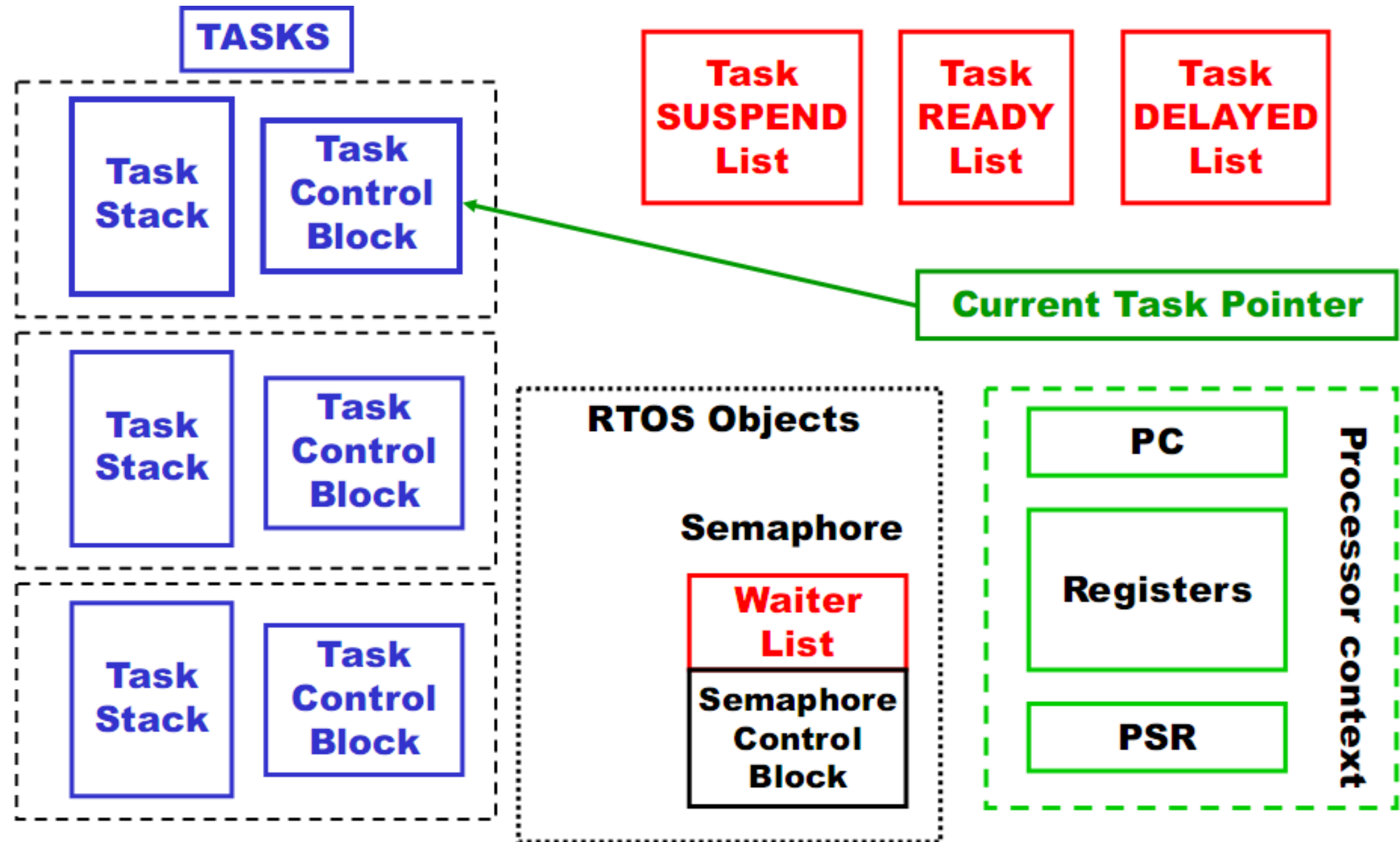
Process = unique execution of a program

- code + data
- multiple processes may share code
- each process has unique data
(CPU registers, stack, memory)
- process defined by its “activation record”

Single Thread vs Multi-Thread

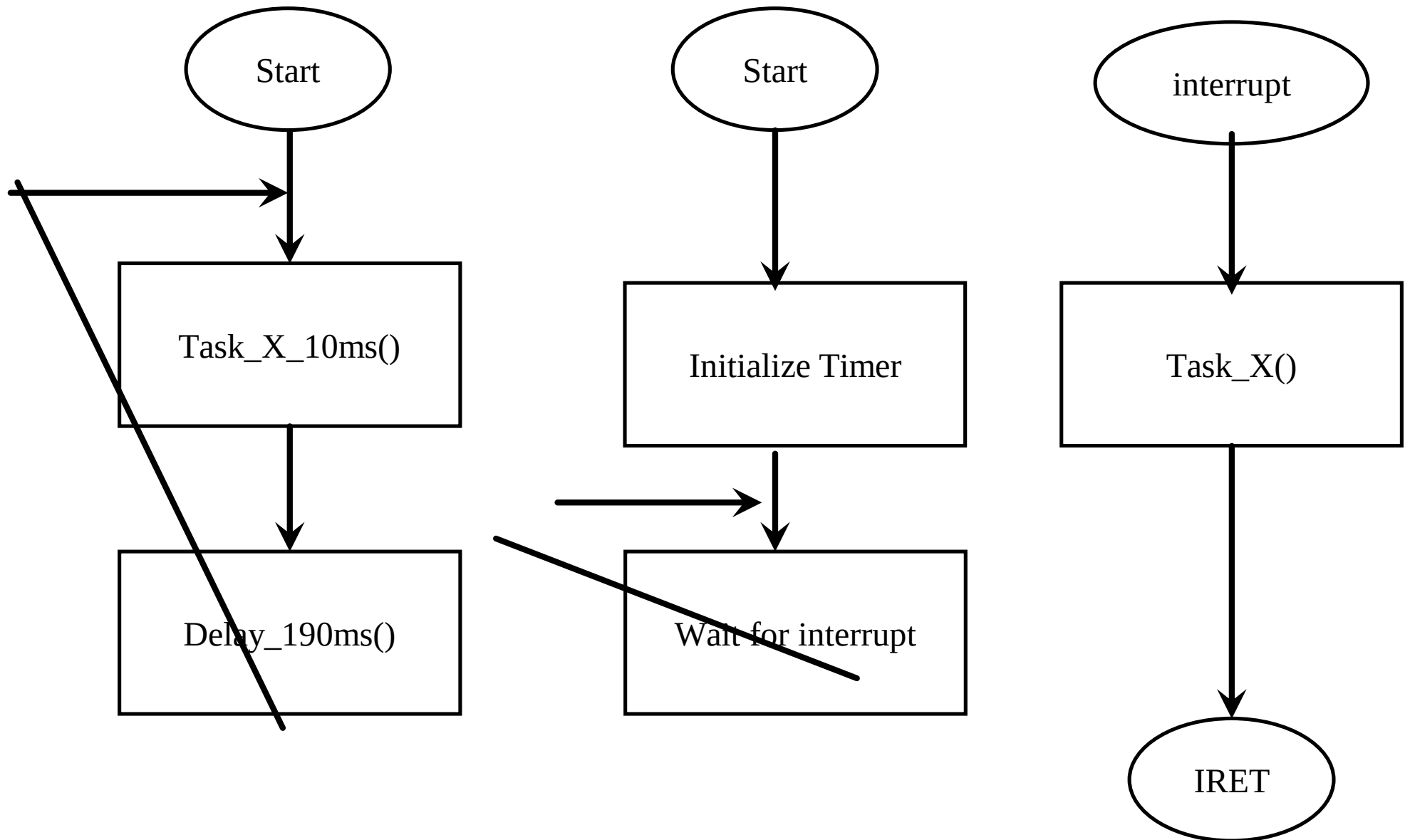


Data Structures in Implementing RTOS

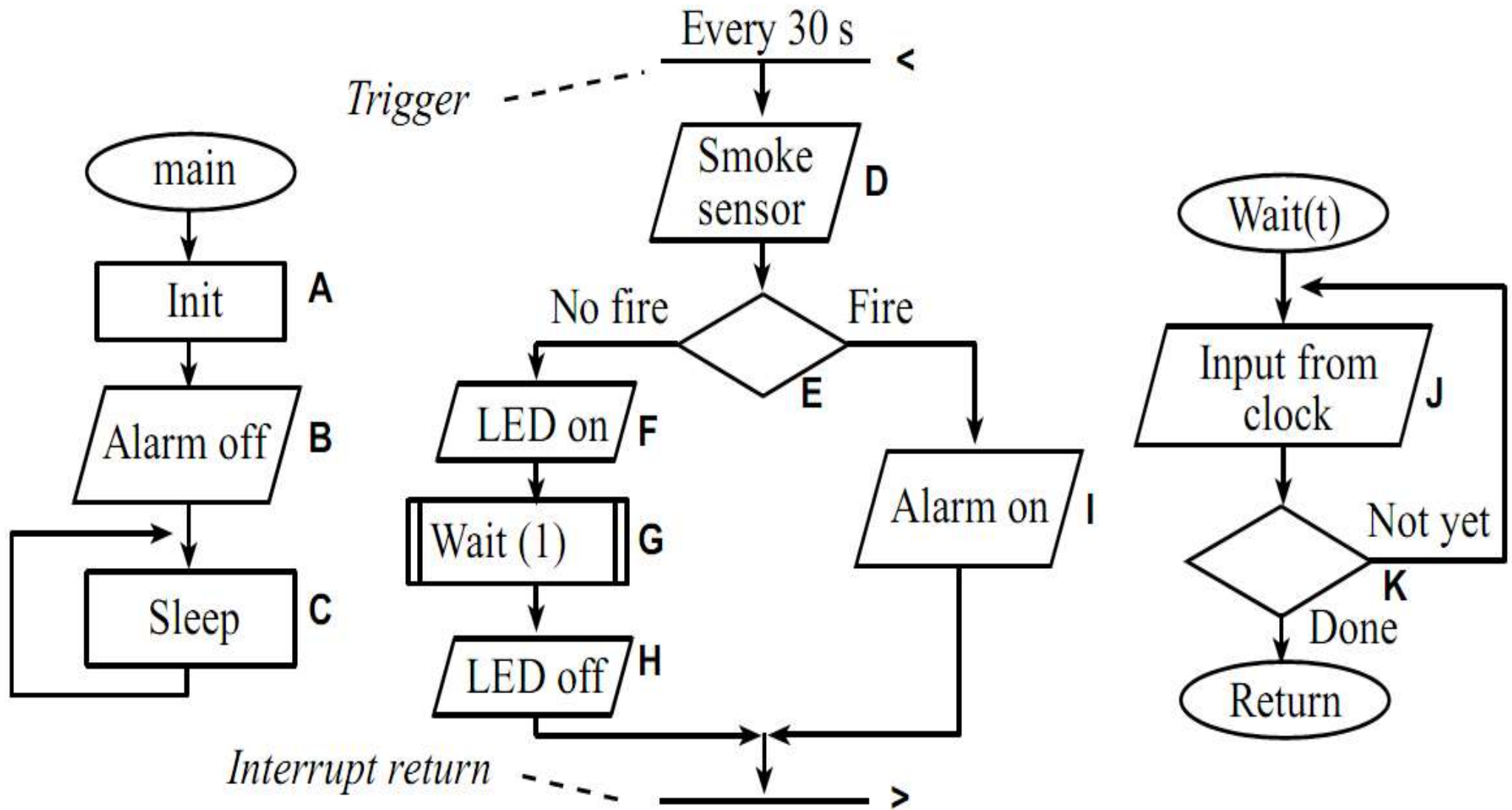


Review of Software Architecture

Review of Architectures in Embedded Systems

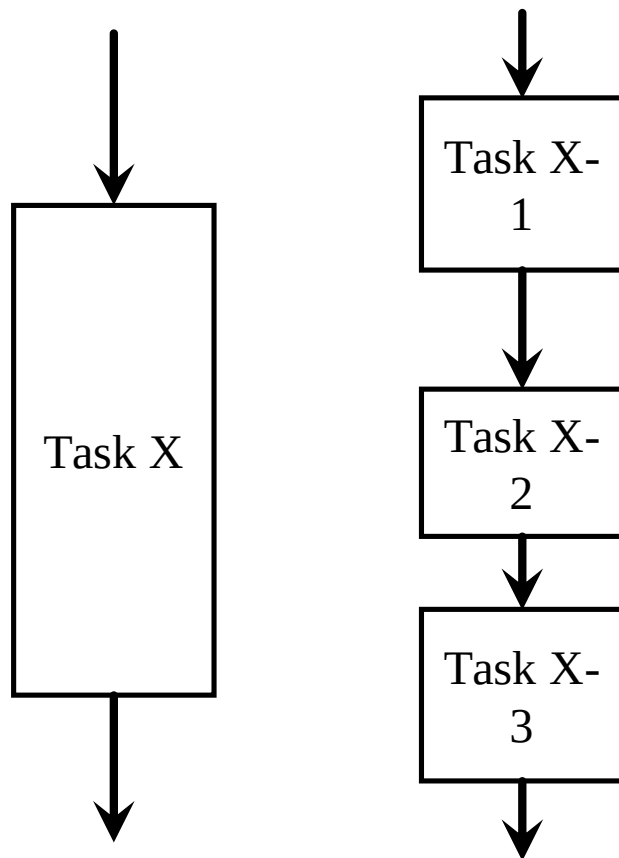


Low Power Design

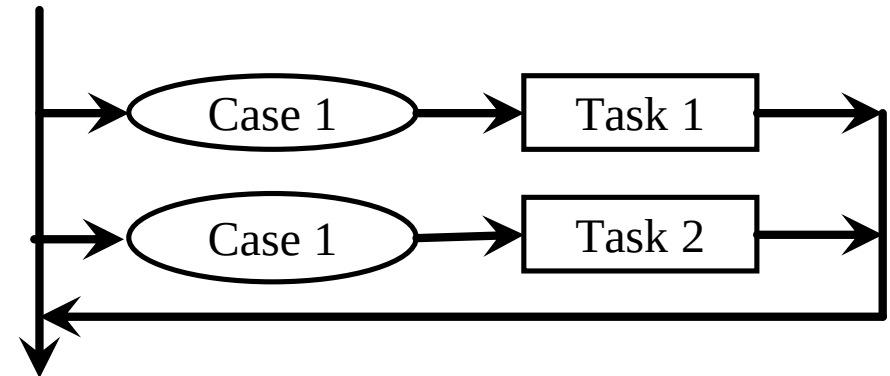


Review of Architectures in Embedded Systems

- Multi-Stage



- Multi-State



Simple Super-Loop Architecture

- A particular limitation with this architecture is that it is very difficult to execute function $X()$ at precise intervals of time (No control over execution timing).
 - *Note: Task/Function $X()$ can consist of several processes (e.g., $p1()$ and $p2()$) => No control over timing of individual processes $p1()$ and $p2()$.*

```
void main(void)
{
    /* Prepare run function X */
    X_Init();

    while(1) /* 'for ever' (Super Loop) */
    {
        X(); /* Run function X */
    }
}
```

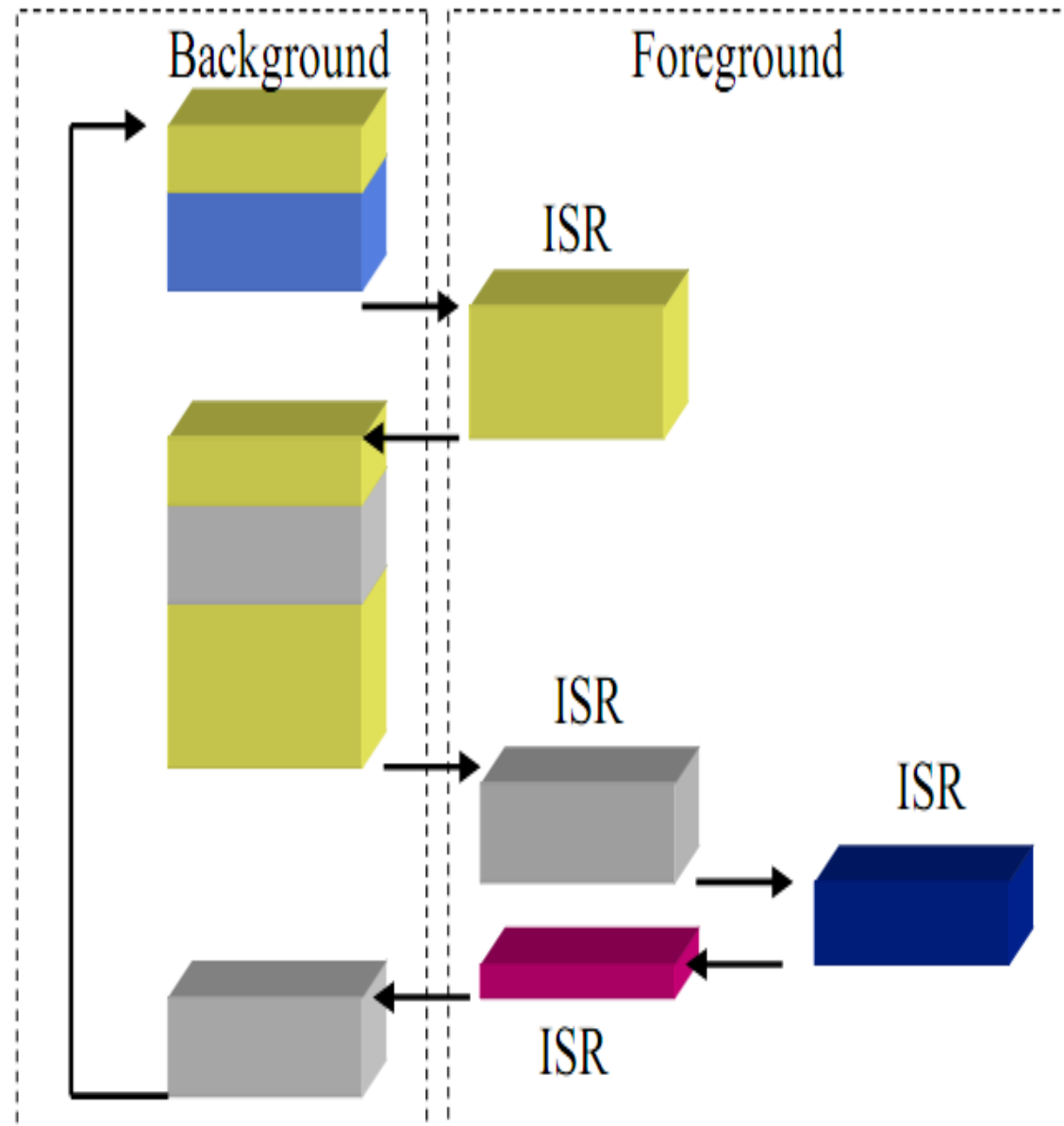
```
void main(void)
{
    Init_System();

    while(1) /* 'for ever' (Super Loop) */
    {
        X(); /* Call the function (10 ms duration) */
        Delay_50ms(); /* Delay for 50 ms */
    }
}
```

This will be fine, if:

1. We know the precise duration of function $x()$, and,
2. This duration never varies.

Super-Loop Execution



Time

```
void main (void) {      // background
  initialization;
  FOREVER {
    read analog inputs;
    read discrete inputs;
    perform monitoring functions;
    perform control functions;
    update analog outputs;
    scan keyboard;
    handle communication requests;
    update display;
  }
}

ISR (void) {             // foreground
  handle asynchronous events;
}
```

Polling

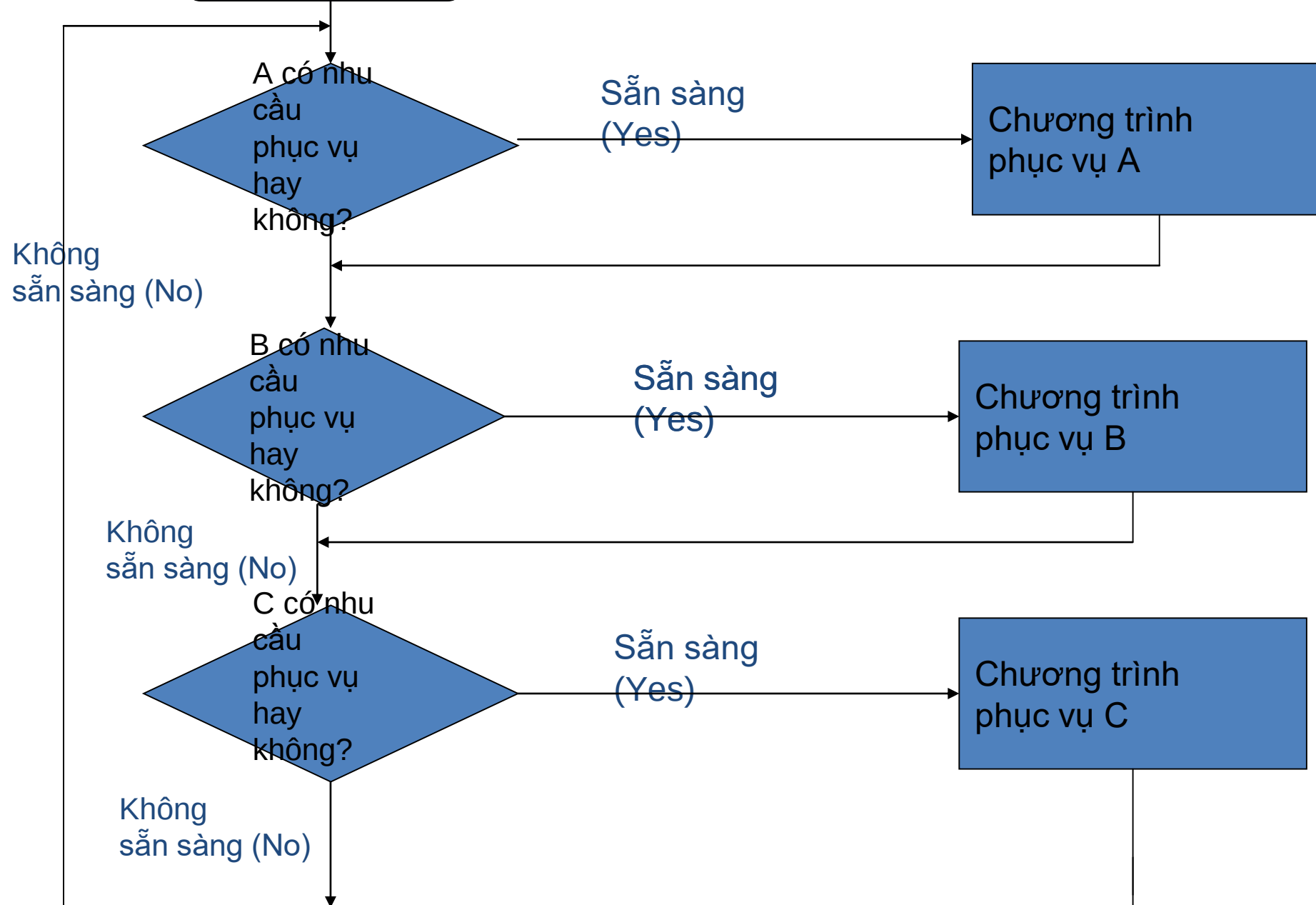
Đặc điểm:

- . Điều khiển vào /ra đơn giản và bằng phần mềm.
- . Thời gian của CPU bị chiếm nhiều để thực hiện chương trình lặp thăm dò.
- . Chỉ nên dùng cho một số thiết bị.
- . Các thiết bị ngoại vi phải đưa ra các bit cờ trạng thái sẵn sàng của thiết bị (trong thanh ghi trạng thái).
- . Phần mềm kiểm tra các bit trạng thái sẵn sàng của thiết bị có trong hệ thống vi xử lý bằng các vòng lặp. Thiết bị nào có trạng thái sẵn sàng thì được phục vụ.
- . Có thể thăm dò tùy ý nhưng phải thăm dò hết các thiết bị có trong có hệ thống và có thể thăm dò theo ưu tiên phục vụ.
- . Dễ dàng thực hiện các sơ đồ ưu tiên(priority schemes).

Non-Prioritized Polling

- Đọc nội dung của thanh ghi trạng thái để kiểm tra.
- Chỉ thăm dò hết các t/b thì vòng thăm dò mới quay trở lại để bắt đầu vòng thăm dò tiếp theo.
- Phụ thuộc vào đặc điểm của t/b ngoại vi mà số vòng nhiều hay ít.
- Phù hợp các hệ thống vi xử lý nhỏ chỉ có một ít t/b ngoại vi và cung thương tương tự nhau về đặc điểm.

Non-Prioritized Polling



Điều khiển vào/ra bằng thăm dò không ưu tiên

Non-Prioritized Polling Code Example

```
while (1) {  
    if (TA_Ready)  
        {TaskA(); }  
  
    if (TB_Ready)  
        {TaskB(); }  
  
    if (TC_Ready)  
        {TaskC(); }  
}
```

Prioritized Polling

- Task readiness checked in order of priority
- Task runs to completion

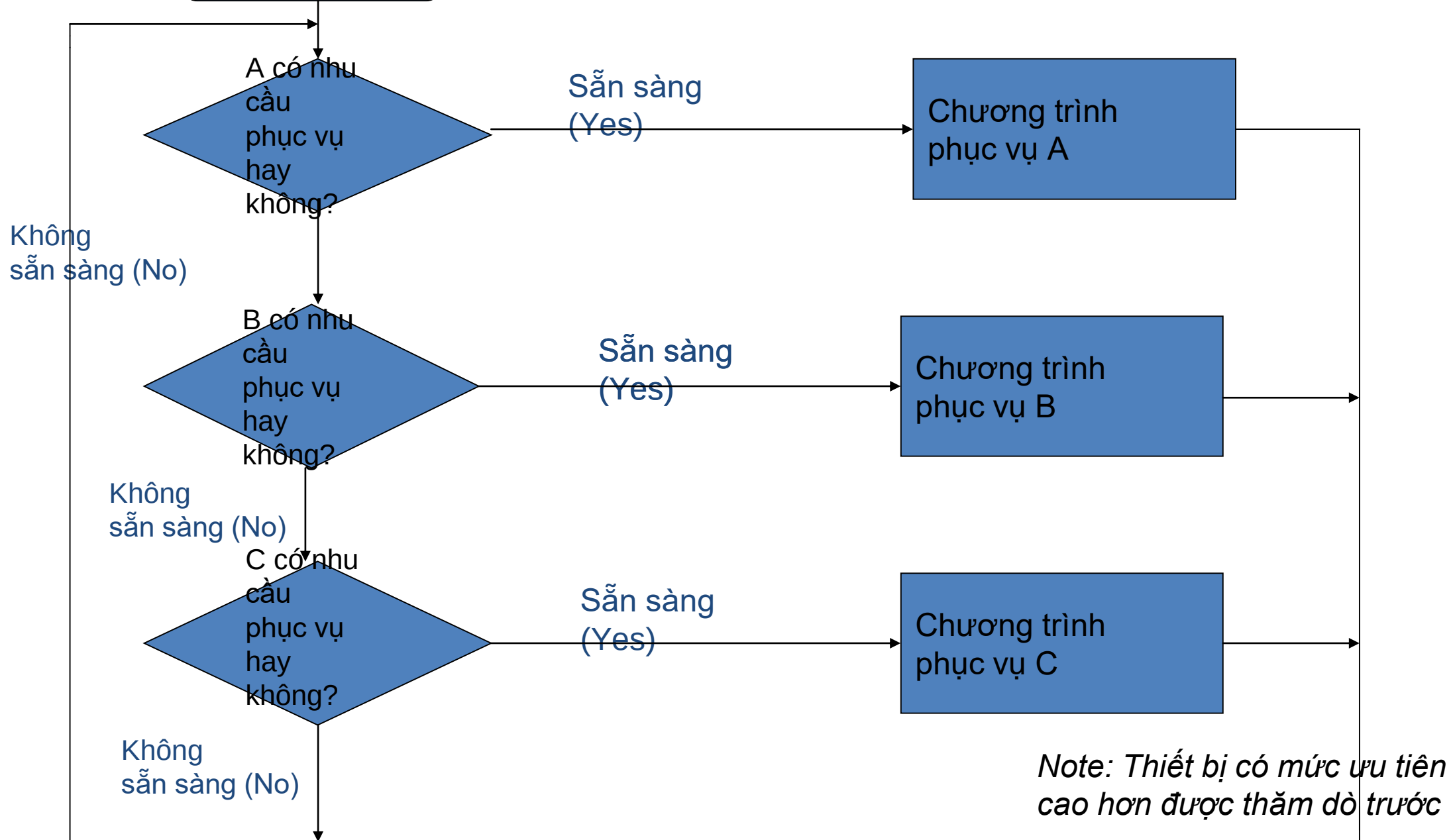
$$T_{response} = \sum_{i < n} N_i T_{Ti} + \max[T_n, T_{n-1}, \dots] + T_{TDn} + T_{cir} + \sum T_{int, srv}$$

higher priority tasks; N_i = #times T_i ready	time to finish a lower priority task	context switch & OS overhead	circuit delays	service interrupts
---	--	---------------------------------------	-------------------	-----------------------

Determining Task Priorities

- ◆ In real-time system assume normal state for all tasks is blocked!
 - ❖ Allows other tasks to execute
 - ❖ A task that runs continuously will prevent lower-priority tasks from executing
- ◆ When a task executes it will delay execution of all lower-priority tasks for as long as it is running.
 - ❖ Determine task priority according to:
 - ↗ a) For how long (max) they execute before blocking. Long execution time => low priority
 - ↗ b) How much delay can they tolerate between being ready-to-run and running. Long delay Ok => lower priority
- ◆ Often task priority does not matter very much. When it does we will learn how to quantify delays later.

Prioritized Polling



Điều khiển vào/ra bằng thăm dò ưu tiên

Prioritized Polling Code Example

```
while (1) {  
    if (TA_Ready)  
        {TaskA(); }  
    else if (TB_Ready)  
        {TaskB(); }  
    else if (TC_Ready)  
        {TaskC(); }  
}
```

Time-slice Scheduling

```
while (1) {  
    wait_for_timer();  
    if (TA_Ready)  
        {TaskA(); }  
    else if (TB_Ready)  
        {TaskB(); }  
    else if (TC_Ready)  
        {TaskC(); }  
}
```

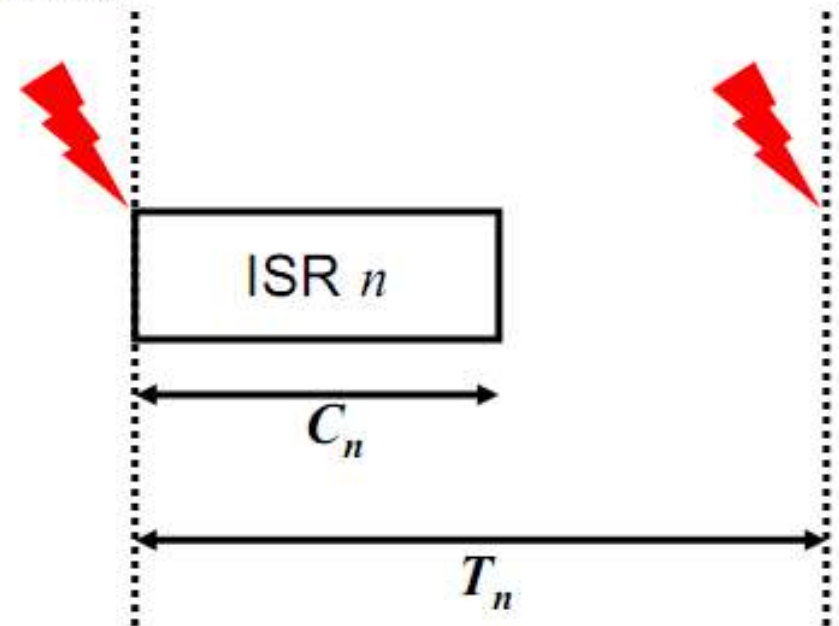
EOS using Interrupts

- Interrupts are the fundamental device which allows sequential computer programs to respond to real-time events
 - Provide a very fast response to a hardware event: Timer timeout, I/O request
- Interrupts require hardware support
 - ALL CPUs provide some way of implementing interrupts
 - Boundary between hardware & software support is variable
 - As a minimum save/switch of PC must be implemented in hardware
- Interrupt processing:
 - Stop code currently executing
 - Branch to Interrupt Service Routine (ISR)
 - Execute ISR
 - Return transparently to previous executing code
- Note that ISR is very different from a task – no state is saved from one invocation to the next
 - Can't implement infinite loop
 - Can't block

Rate Monotonic Analysis for Interrupts

- ◆ Prioritised interrupts provide response to deadlines very similar to prioritised tasks implementing jobs
 - ❖ RMA analysis can be used to determine whether all ISRs will terminate before next ISR invocation

Interrupt priority n
Hardware event initiates computation



Interrupt	Interrupt n occurs	ISR n runs	ISR n returns
Task	Task n becomes ready	Task n runs	Task n blocks

Foreground/Background System Design

- Real-time systems can be implemented without an RTOS using foreground/background design
 - Time-critical computation implemented via interrupt-driven ISR **(foreground)**
 - Non-critical computation implemented in **background** loop
 - Note only one background loop is possible
 - Non-critical hardware can be serviced from background loop using polling
 - Loop length is variable depending on which devices need to be serviced
 - Worst-case loop length determines latency for polled devices
- Advantages: Simple and more efficient than RTOS
- Disadvantages
 - More computation must be implemented in ISR – coding more complex & difficult to debug
 - Can't use tasks to simplify & modularise design

Polling from Timer interrupts

- ◆ **Interrupt-driven paradigm locates computation in specific ISRs executed in response to hardware events**
 - ❖ Interrupt-driven computation in ISR triggered by hardware event
 - ❖ Response time is limited by CPU-determined interrupt time + execution time of any higher or equal priority interrupts
- ◆ **Timer-interrupt ISR polls multiple devices at regular intervals**
 - ❖ provides guaranteed response with less hardware overhead than device-specific hardware interrupts
 - ❖ Simpler to code than multiple separate ISRs
- ◆ **Cf time-interrupt polling & background loop polling**
 - ❖ Background loop also implements polled computation – but time between successive polls cannot easily be determined.

```
BackgroundLoop()
{
    for (;;) {
        if (device A is ready) [ Service Device A ];
        if (device B is ready) [ Service Device B ];
    }
}

Timer1_ISR() // device C or D
{
    if (device C is ready) [ Service Device C ];
    if (device D is ready) [ Service Device D ];
    [ update system clock ]
}

DeviceE_ISR()
{
    [ Service Device E ]; /*NB polling not needed*/
}
```

Response-time:

Timer ISR polling vs background loop

- Can use one or more timer ISRs running at different speeds –e.g. 1ms and 10ms – to implement half-way house between polled & interrupt-driven paradigms
 - 1ms interrupt runs higher priority than 10ms interrupt
 - 1ms interrupt provides response-time (latency) of 1ms worst case, but total length of ISR must be $\ll 1\text{ms}$
 - 10ms interrupt provides response-time of 10ms worst case. Total length of ISR must be $\ll 10\text{ms}$. Note that 10ms ISR execution time is effectively slowed down by CPU utilisation due to 1ms interrupt (and other high priority interrupts)
- Background loop can be used to implement polled computation which is not time-critical.
 - Polling time for background loop is variable and depends on overall loop period.

Simple EOS using Timer-based Interrupts

- Timer-based interrupts as the core of an embedded OS

```
#include "Main.H"
#include "Port.H"
#include "Simple_EOS.H"

#include "X.H"

/* ----- */

void main(void)
{
    /* Prepare for dummy task */
    X_Init();

    /* Set up simple EOS (60 ms tick interval) */
    sEOS_Init_Timer2(60);

    while(1) /* Super Loop */
    {
        /* Enter idle mode to save power */
        sEOS_Go_To_Sleep();
    }
}
```


Simple EOS using Timer-based Interrupts

```
/*-----*/
Simple EOS.C (v1.00)
/*-----*/
#include "Main.H"
#include "Simple_EOS.H"

/* Header for dummy task */
#include "X.H"

/*-----*/
```

```
sEOS_ISR()

Invoked periodically by Timer 2 overflow:
see sEOS_Init_Timer2() for timing details.

/*-----*/
sEOS_ISR() interrupt INTERRUPT_Timer_2_Overflow
{
    /* Must manually reset the T2 flag */
    TF2 = 0;

    /*===== USER CODE - Begin ===== */

    /* Call dummy task here */
    X();

    /*===== USER CODE - End ===== */
}
```

```
/*-----*/
sEOS_Go_To_Sleep()
```

This operating system enters 'idle mode' between clock ticks to save power. The next clock tick will return processor to the normal operating state.

```
/*-----*/
void sEOS_Go_To_Sleep(void)
{
    PCON |= 0x01;    /* Enter idle mode (generic 8051 version) */
}
```

```
/*-----*/
sEOS_Init_Timer2()

/*-----*/
void sEOS_Init_Timer2(const tByte TICK_MS)
{
    tLong Inc;
    tWord Reload_16;
    tByte Reload_08H, Reload_08L;

    /* Timer 2 is configured as a 16-bit timer,
       which is automatically reloaded when it overflows */
    T2CON = 0x04;    /* Load T2 control register */

    /* Number of timer increments required (max 65536) */
    Inc = ((tLong)TICK_MS * (OSC_FREQ/1000)) /
          (tLong)OSC_PER_INST;

    /* 16-bit reload value */
    Reload_16 = (tWord) (65536UL - Inc);

    /* 8-bit reload values (High & Low) */
    Reload_08H = (tByte) (Reload_16 / 256);
    Reload_08L = (tByte) (Reload_16 % 256);

    /* Used for manually checking timing (in simulator) */
    /*P2 = Reload_08H; */
    /*P3 = Reload_08L; */

    TH2 = Reload_08H;    /* Load T2 high byte */
    RCAP2H = Reload_08H; /* Load T2 reload capt. reg h byte */
    TL2 = Reload_08L;    /* Load T2 low byte */
    RCAP2L = Reload_08L; /* Load T2 reload capt. reg l byte */

    /* Timer 2 interrupt is enabled, and ISR will be called
       whenever the timer overflows. */
    ET2 = 1;

    /* Start Timer 2 running */
    TR2 = 1;

    EA = 1;    /* Globally enable interrupts */
}
```

Simple EOS using Timer-based Interrupts

```
/*-----*  
  X.C (v1.00)  
  Dummy task to introduce sEOS.  
-----*/  
  
#include "X.H"  
  
void X_Init(void)  
{  
  /* Dummy task init... */  
}  
  
/*-----*  
  X() Dummy task called from sEOS ISR.  
-----*/  
  
void X(void)  
{  
  /* Dummy task... */  
}  
  
/*-----*  
  ---- END OF FILE ----
```

Comments on Simple EOS (sEOS)

- Advantages:
 - This sEOS imposes a very low processor load and is rather flexible and useful for simple applications
- Disadvantages:
 - *Task/Function X() can consist of several processes (e.g., p1() and p2()) => No control over timing of individual processes => use multiple-timer based interrupt or RTOS*
 - Unlike a traditional RTOS, this sEOS becomes part of the application itself, rather than forming a separate code layer => make the code less abstract and clarified
 - Hard to apply to systems of complexity with many timing deadlines and constraints, priorities.

Simple EOS using Multiple-Timer Based Interrupts

- Each task has its own function and own set of processes
 - E.g., Task/Function X1() has processes p1() and p2().
- Each task has its own timer.
 - E.g., can allocate timer 1 for task X1 & timer 2 for task X2
 - May not have enough timers to implement all the rates.
 - Complexity and unreliability due to the presence of multiple interrupts

```
void X1(){  
    /* rate T1 for task X1() */  
    p1();  
    p3();  
}  
  
void X2(){  
    /* rate T2 for task X2() */  
    p2();  
    p4();  
    p5();  
}
```

Simple EOS using Virtual Multiple-Timer-Based Interrupts

- Instead of using 'Real' multiple timers, use a software count to divide the timer (i.e., create virtual multiple timers)
- Only works for clean multiples of the timer period.
- Incur overhead and inaccuracy due to the operation for creating virtual multiple timers (e.g., checking instructions or updating the variable countx).

```
int count2, count3 = 0;
void X(){
    /* rate T1 for p1() */
    p1();
    /* rate T2 for p2() */
    if (count2 >= 2) {
        p2();
        count2 = 0;
    } else count2++;
    /* rate T3 for p3() */
    if (count3 >= 5) {
        p3();
        count3 = 0;
    } else count3++;
}
```


Multi-State System

- **Multi-State (Timed)**

In a multi-state (timed) system, the transition between states will depend **only** on the passage of time.

For example, the system might begin in State A, repeatedly executing `FunctionA()`, for ten seconds. It might then move into State B and remain there for 5 seconds, repeatedly executing `FunctionB()`. It might then move back into State A, *ad infinitum*.

A basic traffic-light control system might follow this pattern.

- **Multi-State (Input / Timed)**

This is a more common form of system, in which the transition between states (and behaviour in each state) will depend both on the passage of time **and** on system inputs.

For example, the system might only move between State A and State B if a particular input is received within X seconds of a system output being generated.

The autopilot system discussed at the start of this seminar might follow this pattern, as might a control system for a washing machine, or an intruder alarm system.

Please also note that the system states will usually be represented by means of a `switch` statement in the operating system ISR.

Multi-State (Timed) Example: Traffic light sequencing

In the code, we will represent these states as follows:

```
/* Possible system
typedef enum {RED, RED_AND_AMBER, GREEN, AMBER}
eLight_State;
```

We will store the time to be spent in each state as follows:

```
/* (Times are in
seconds) */
#define RED_DURATION 20
#define RED_AND_AMBER_DURATION 5
#define GREEN_DURATION 30
#define AMBER_DURATION 5

void
main(void)
{
    /* Prepare to run traffic
    TRAFFIC_LIGHTS_Init(RED
    );
    /* Set up simple EOS (50 ms
    EOS_Init_Timer2(
    50);
    while(1) /* Super Loop
        {
            /* Enter idle mode to save
            EOS_Go_To_Sleep(
            );
        }
}
```

```
sEOS_ISR() interrupt INTERRUPT_Timer_2_Overflow
{
    TF2 = 0;
    TRAFFIC_LIGHTS_Update();
}

void TRAFFIC_LIGHTS_Update(void)
{
    switch (Light_state_G){

        case RED: {

            Red_light = ON;
            Amber_light = OFF;
            Green_light = OFF;

            if(++Time_in_state == RED_DURATION){

                Light_state_G = RED_AND_AMBER;
                Time_in_state = 0;
            }

            break; }

        ...

        case AMBER: {

            Red_light = OFF;
            Amber_light = ON;
            Green_light = OFF;

            if(++Time_in_state == AMBER_DURATION){

                Light_state_G = RED;
                Time_in_state = 0; }

            break; }

    }
}
```

Multi-State (Input/Timed) Example: Washing machine

```

/*-----*/
void WASHER_Update(void)
/* Call once per switch */
(System_state_G) {
case
    INIT:
    /* For demo purposes
    Debug_port = (tByte)
    System_state_G;
    /* Set up initial
    state Motor is
    WASHER_Control_Motor(OFF
    );
    /* Pump is
    WASHER_Control_Pump(OFF)
    ;
    /* Heater is
    WASHER_Control_Water_Heater(OFF);
    /* Valve is
    WASHER_Control_Water_Valve(OFF);
    /* Wait (indefinitely) until START
    is pressed
    if (WASHER_Read_Start_Switch() !=
        1) {
        return
        ; }

    /* Start switch pressed ->
    read the selector dial */
    Program_G
    WASHER_Read_Selector_Dial();
    /* Change
    System_state_G =
    START; break;
    }
    .....

```

```

case START: {

    /* For demo purposes only */
    Debug_port = (tByte) System_state_G;

    /* Lock the door */
    WASHER_Control_Door_Lock(ON);

    /* Start filling the drum */
    WASHER_Control_Water_Valve(ON);

    /* Release the detergent (if any) */
    if (Detergent_G[Program_G] == 1) {
        WASHER_Control_Detergent_Hatch(ON);
    }

    /* Ready to go to next state */
    System_state_G = FILL_DRUM;
    Time_in_state_G = 0;

    break;
}
.....

case HEAT_WATER:
{
    /* For demo purposes only */
    Debug_port = (tByte) System_state_G;

    /* Remain in this state until water is hot NOTE: Timeout
    facility included here */
    if (++Time_in_state_G >= MAX_WATER_HEAT_DURATION){

        /* Should have warmed the water by now... */
        System_state_G = ERROR;
    }

    /* Check the water temperature */

```

Comments on Multi-State System

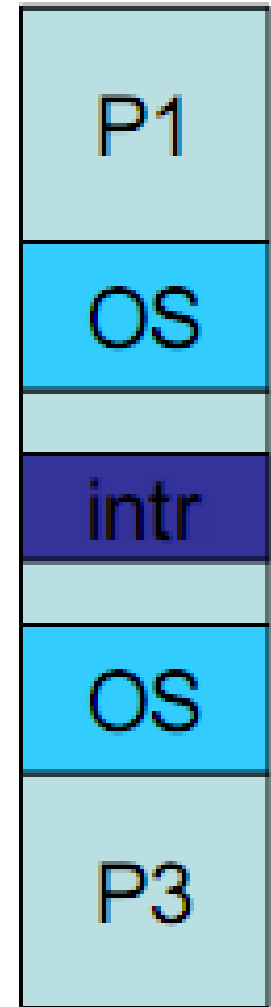
- The use of the multi-state (timed and input/timed) systems in conjunction with an OS (like the aforementioned sEOS) is widespread in embedded applications thanks to the very flexibility.

Summary: Interrupts in RTOS

- Simplest real-time systems are implemented using the foreground/background model using interrupts and background loop.
- Interrupts can be used to provide guaranteed latency & response-time
 - Interrupts have priority which determines precedence
 - Interrupt priorities are managed by device-dependent hardware or software.
 - All CPUs allow individual interrupt sources to be switched on or off which in principle allows software control of priorities
 - In practice for speed prioritised interrupts are managed by special hardware in big systems
- I/O operations can be performed:
 - From hardware-specific interrupt
 - From timer interrupt (polled)
 - From background loop (polled)
- RMA can be used to determine whether prioritised interrupt deadlines are met
 - More precise calculation must add interrupt entry and return times to ISR time when calculating RMA compute time C_i
 - Deadline is met if interrupt response-time is less than time between interrupts

Summary: Guidelines of Interrupt Use

- Interrupt overhead
 - Interrupts take time away from processes
 - Other event processing may be masked during interrupt service routine (ISR)
 - Perform minimum work possible in the interrupt handler
- Device processing structure
 - Interrupt service routine (ISR) performs minimal I/O.
 - Get register values, put register values
 - Interrupt service process/thread performs most of device function.



How To Implement Long Interrupts

- Interrupt handling sometimes needs to perform lengthy tasks. This problem is resolved by splitting the interrupt handler into two halves:
- Top half responds to the interrupt
 - The one registered to request_irq
 - Saves data to device-specific buffer and schedules the bottom half
 - Current interrupt disabled, possibly all disabled.
 - Runs in interrupt context, not process context. Can't sleep.
 - Acknowledges receipt of interrupt.
 - Schedules bottom half to run later.
- Bottom half is scheduled by the top half to execute later
 - With all interrupts enabled
 - Wakes up processes, starts I/O operations, etc.
 - Runs in process context with interrupts enabled.
 - Performs most work required. Can sleep.
 - Ex: copies network data to memory buffers.

How To Implement Long Interrupts

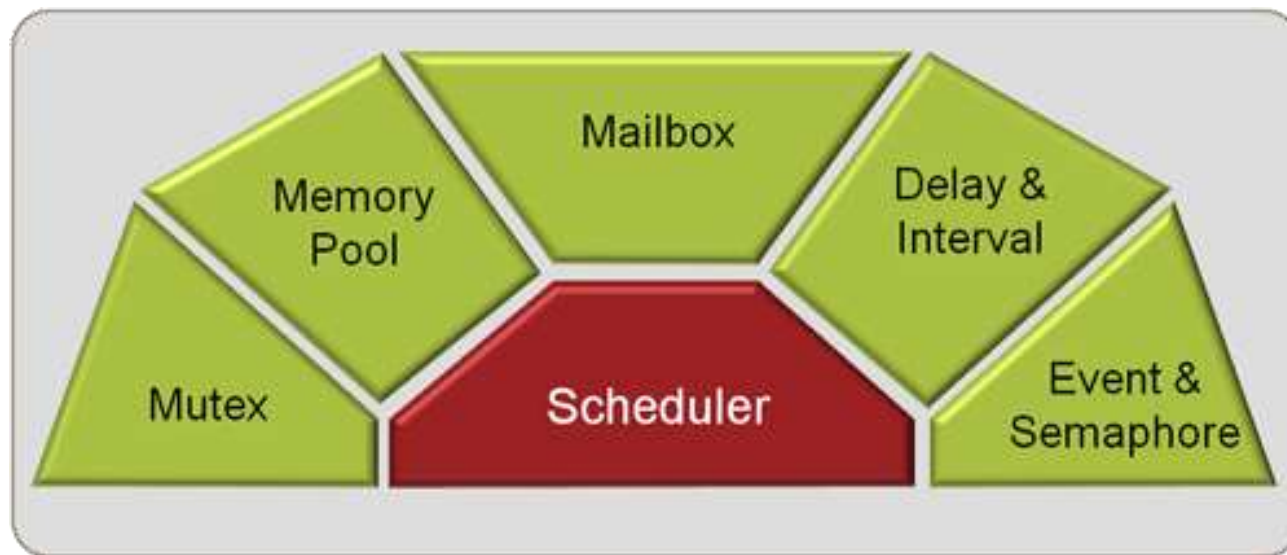
- Three mechanisms may be used to implement bottom halves
 - SoftIRQs
 - Have strong locking requirements
 - Only used of performance sensitive subsystems –networking, SCSI, etc.
 - Reentrant
 - Tasklets
 - Built on top of SoftIRQs
 - Should not sleep
 - Cannot run in parallel with itself
 - Can run in parallel with other tasklets on SMP systems
 - Guaranteed to run on the same CPU that first scheduled them
 - Workqueues
 - Can sleep
 - Cannot copy data to and from user space

Dos and Don'ts of Interrupt Handlers

- It's a programming offense if your interrupt context code goes to sleep. Interrupt handlers cannot relinquish the processor by calling sleepy functions such as `schedule_timeout()`.
- For protecting critical sections inside interrupt handlers, you can't use mutexes because they may go to sleep. Use spinlocks instead, and use them only if you must.
- Interrupt handlers are supposed to get out of the way quickly but are expected to get the job done. To circumvent this Catch-22, interrupt handlers split their work into two halves: top (slim) and bottom (fat).
- You do NOT need to design interrupt handlers to be reentrant. When an interrupt handler is running, the corresponding IRQ is disabled until the handler returns.
- Interrupt handlers can be interrupted by handlers associated with IRQs that have higher priority. You can prevent this nested interruption by specifically requesting the kernel to treat your interrupt handler as a fast handler.

SCHEDULER IMPLEMENTATION IN RTOS

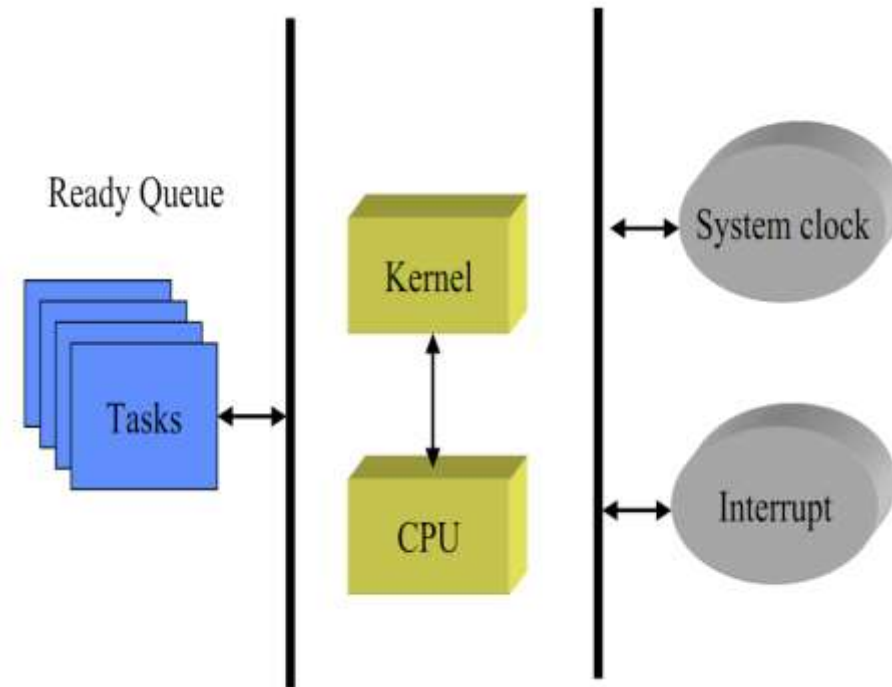
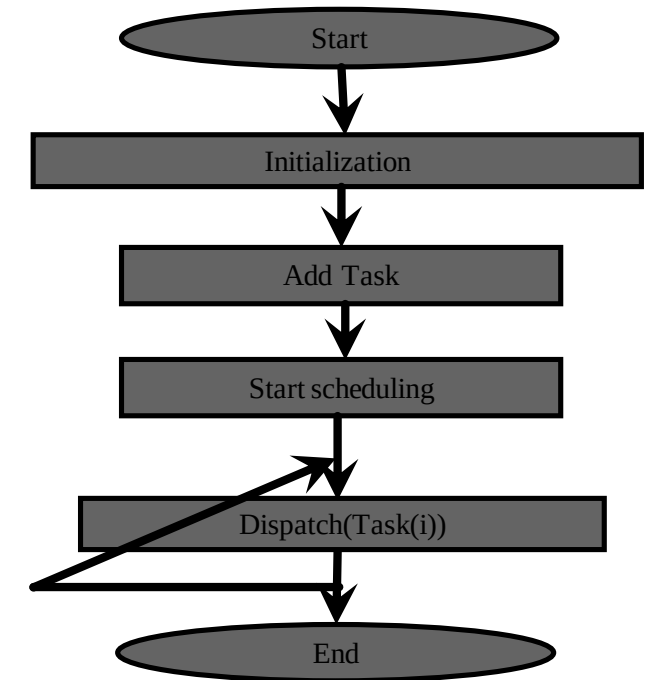
Scheduler Structure



Scheduler

- **The scheduler:**

- One of the jobs of the OS is to run the scheduler
- The scheduler is a piece of code that implements certain policy to pick up the next process to run
 - Typically involves a context switching
- A scheduler can be viewed as a simple operating system that allows tasks to be called periodically or on a one-shot basis
- A scheduler can be viewed as a single timer interrupt between many different tasks



Basic Structure of Schedulers

✓ Scheduler data structure and Tasks array

✓ Initialization function

✓ Update function

✓ This function does not execute tasks
It only determines when tasks are
run and which task is to be run and
sets flags

✓ Add task function

✓ Dispatch function

✓ Execute the eligible task.

✓ Start function

✓ Delete function

```
typedef struct
{
    void (code * pTask)(void);
    tWord Delay;
    tWord Period;
    tByte RunMe;
    tByte Co-op;
} sTask;

sTask SCH_tasks_G[SCH_MAX_TASKS];

SCH_Add_Task(Task_Name, Initial_Delay, Period, Co-op)
```

Structure of Schedulers

Độ trễ thời gian (đơn vị Tick) trước khi Task được thực hiện lần đầu tiên. Nếu giá trị này =0 thì task này sẽ được thực hiện ngay sau khi bộ Scheduler bắt đầu.

Co-op =1 nếu là Task không ưu tiên
Co-op=0 nếu là Task ưu tiên.

SCH_Add_Task(Task_Name, Initial_Delay, Period, Co-op)

Tên của hàm mà chúng ta muốn lập lịch.

Chu kỳ của Task được thực thi. Nếu giá trị này bằng 0 thì task này chỉ được thực hiện duy nhất một lần.

Example of Application using Schedulers

```
/*-----*/

void main(void)                                     // Start the scheduler
{
    SCH_Start();

    // Set up the scheduler

    SCH_Init_T2();

    // Prepare for the 'Flash_LED' task

    LED_Flash_Init();

    // Add the 'Flash LED' task (on for ~1000 ms, off for ~1000 ms)

    // - timings are in ticks (1 ms tick interval)

    // (Max interval / delay is 65535 ticks)

    SCH_Add_Task(LED_Flash_Update, 0, 1000);

    /*-----*/
    void SCH_Update(void) interrupt INTERRUPT_Timer_2_Overflow
    {
        // Update the task list

        ...}
}
```

Add function

- Add tasks to the task array to ensure the added tasks will be called at requested times.

Độ trễ thời gian (đơn vị Tick) trước khi Task được thực hiện lần đầu tiên. Nếu giá trị này =0 thì task này sẽ được thực hiện ngay sau khi bộ Scheduler bắt đầu.

Co-op=1 nếu là Task không ưu tiên
Co-op=0 nếu là Task ưu tiên.

`SCH_Add_Task(Task_Name, Initial_Delay, Period, Co-op)`

Tên của hàm mà chúng ta muốn lập lịch.

Chu kỳ của Task được thực thi. Nếu giá trị này bằng 0 thì task này chỉ được thực hiện duy nhất một lần.

```
tByte SCH_Add_Task(void (code * pFunction)(),
const tWord DELAY,
const tWord PERIOD)
{
    tByte Index = 0;

    // First find a gap in the array (if there is one)
    while ((SCH_tasks_G[Index].pTask != 0) && (Index < SCH_MAX_TASKS))
    {
        Index++;
    }

    // Have we reached the end of the list?
    if (Index == SCH_MAX_TASKS)
    {
        // Task list is full
        //
        // Set the global error variable
        Error_code_G = ERROR_SCH_TOO_MANY_TASKS;

        // Also return an error code
        return SCH_MAX_TASKS;
    }

    // If we're here, there is a space in the task array
    SCH_tasks_G[Index].pTask = pFunction;
    SCH_tasks_G[Index].Delay = DELAY;
    SCH_tasks_G[Index].Period = PERIOD;
    SCH_tasks_G[Index].RunMe = 0;

    return Index; // return position of task (to allow later deletion)
}
```


Common RTOS Design Decisions

- Write RTOS in C
 - Use C preprocessor to switch features on/off
 - Use C macros for efficient implementation of hardware-dependent primitives
- Use global memory space –ignore hardware memory management
 - Appropriate for small & medium-sized embedded systems
- Implement RTOS objects, tasks, semaphores, etc, via control blocks as C structures:
 - TCB – task control block
 - SCB – semaphore control block
- Use a single periodic timer interrupt "clock tick" to provide RTOS timing functions
 - Aids portability – machine-dependent code is localised
- Use dynamic task creation
- Use priority-based preemptive scheduling
- Provide (at least) semaphores, message queues

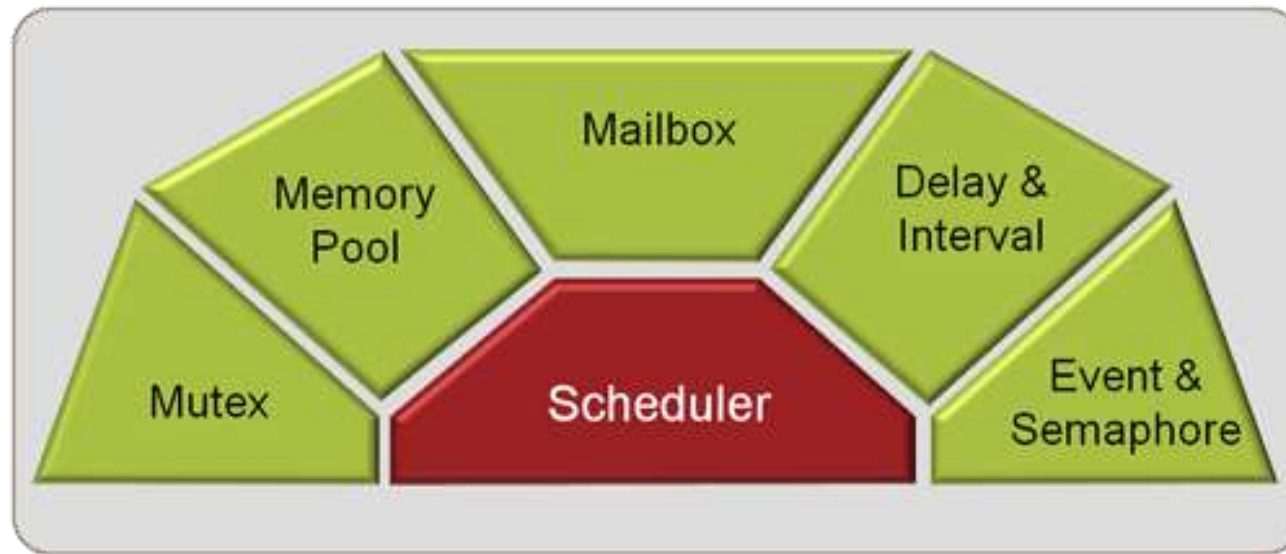
RT and RTOS Conclusion

Analysis

- RT introduces a wide variety of problems
- RMA is best quantitative technique to ensure correctness- works with prioritised system
- Liveness problems are a big issue in RT systems which can only be addressed through careful attention at application design stage
 - Deadlock – crucial but straightforward
 - Starvation – easy to mend but more difficult to detect
 - Livelock – even more difficult to detect
 - Priority inversion – must be fully understood. A variety of RTOS-specific solutions are available

Implementation

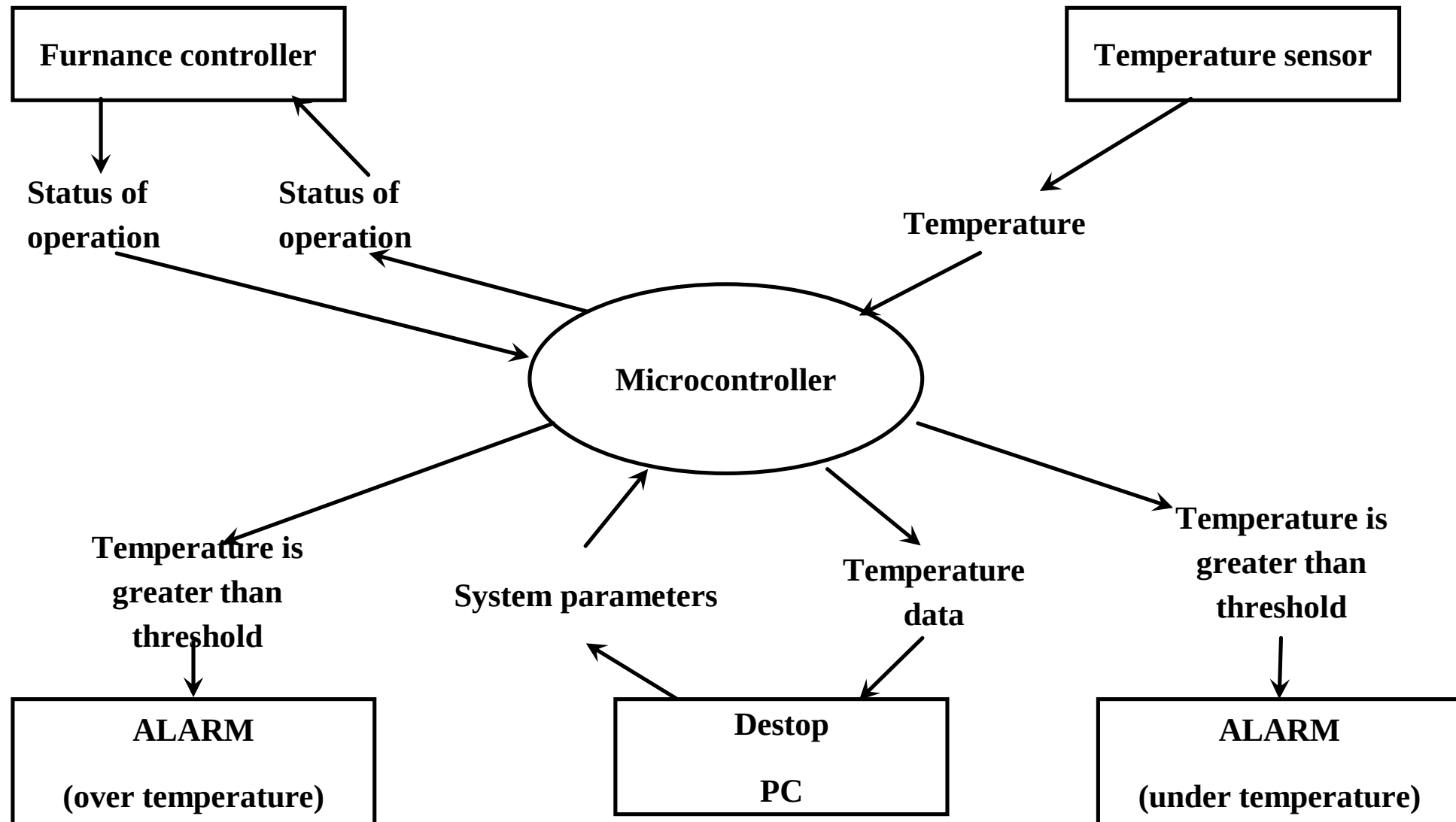
- RTOS implementation is still relatively immature, with competing methods for the various internal functions
- Unlike other OS, scalability is essential, with ability to run on both very resource-constrained systems and relatively large systems a merit
- To achieve this RTOS must be configurable with code size dependent on selected feature-set
- RTOS can relatively easily be ported to new architectures – port depends on details of CPU, C compiler, and assembler.
 - It is sometimes possible to use “inline assembly code” embedded in C functions and avoid using a separate

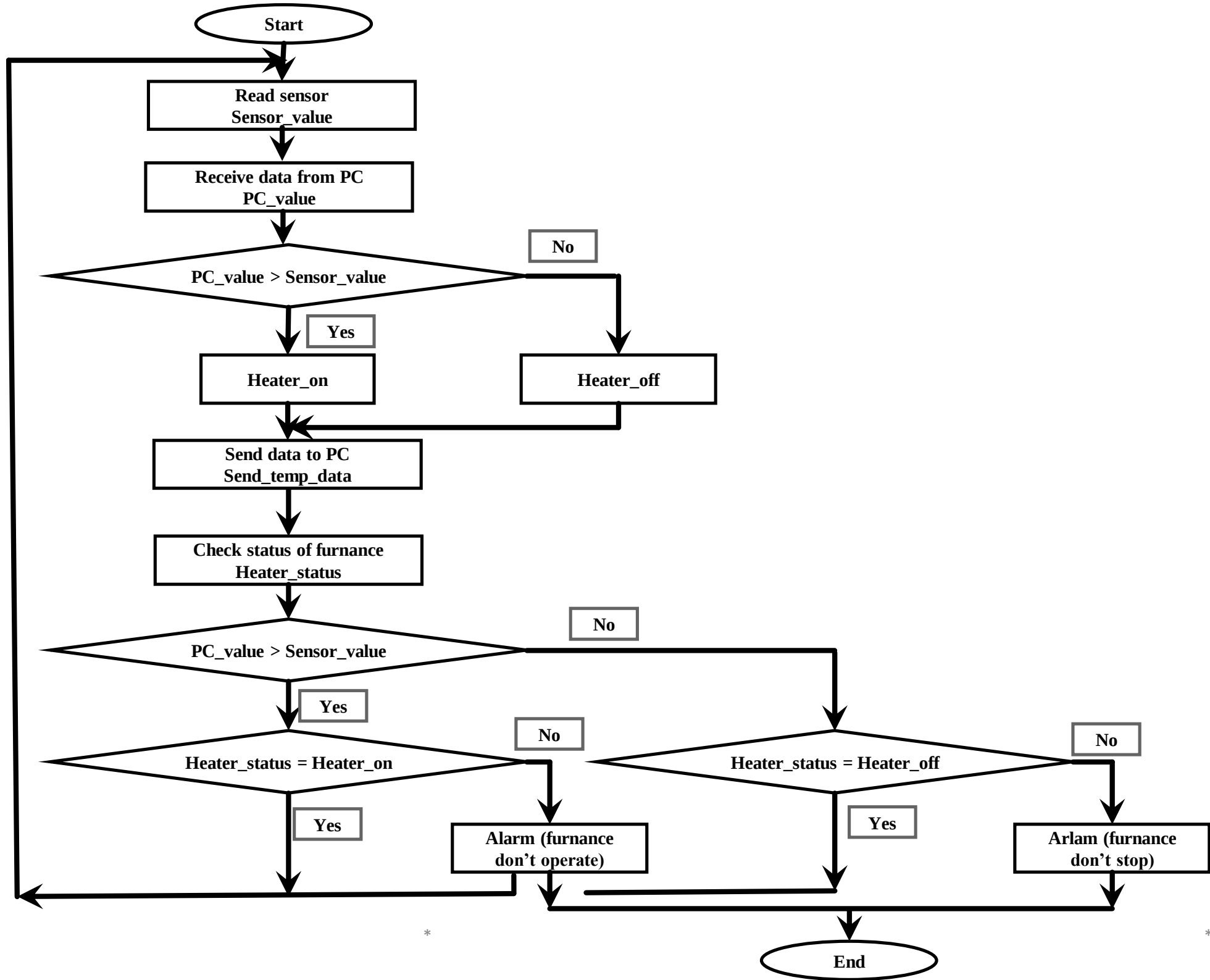


EXAMPLES OF SCHEDULING APPLICATIONS

Applications Examples

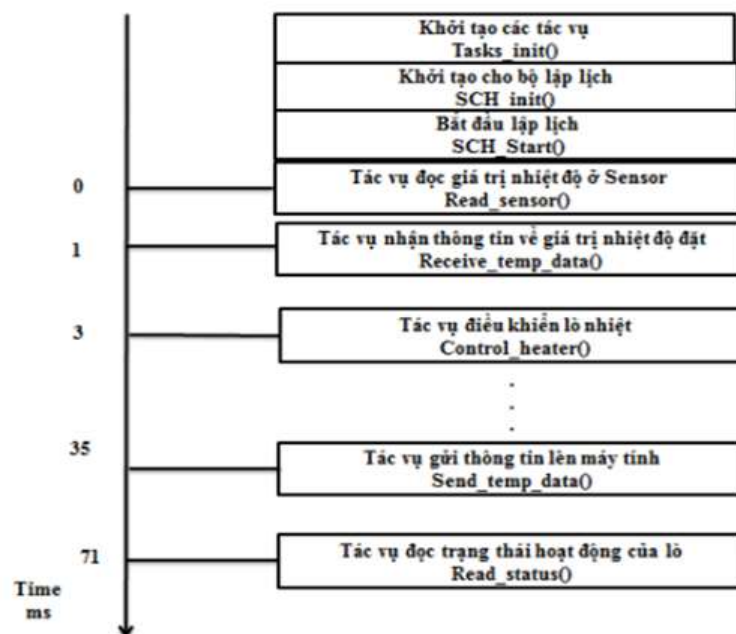
- **1. Temperature-monitoring and control system for a metal furnance**





Tasks

- void Read_sensor(void){...}
- void Receive_temp_data(void) {...}
- void Control_heater() {...}
- void Send_temp_data(void) {...}
- void Read_status_heater() {...}



```
int main (void)
{
    STM32_init();

    SCH_Add_Task(Read_sensor, 0, 200,1);

    SCH_Add_Task(Receive_temp_data, 1, 30,1);

    SCH_Add_Task(Control_heater, 3, 4,0);

    SCH_Add_Task(Send_temp_data, 35, 200,1);

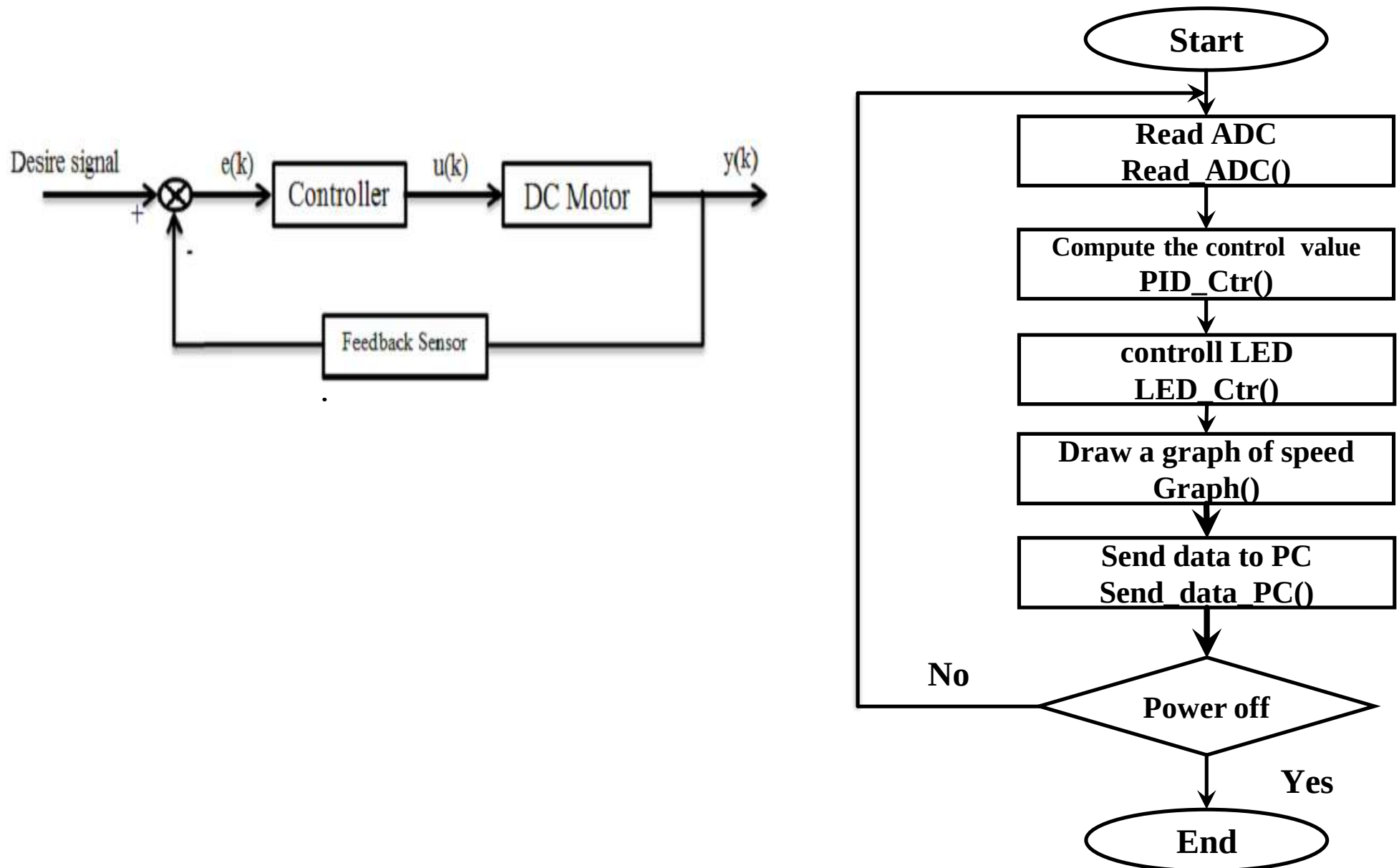
    SCH_Add_Task(Read_status, 71, 1000,1);

    SCH_Start();

    While(1)
    {
        Dispatch();
    }
}
```


- Steps to implementing a system by using Scheduler
 - Analyse operation of system
 - Algorithm diagram
 - Determine tasks that need to be executed
 - Select periods for tasks
 - Select type of scheduler
 - Implementation

2. Controller of DC motor by PWM

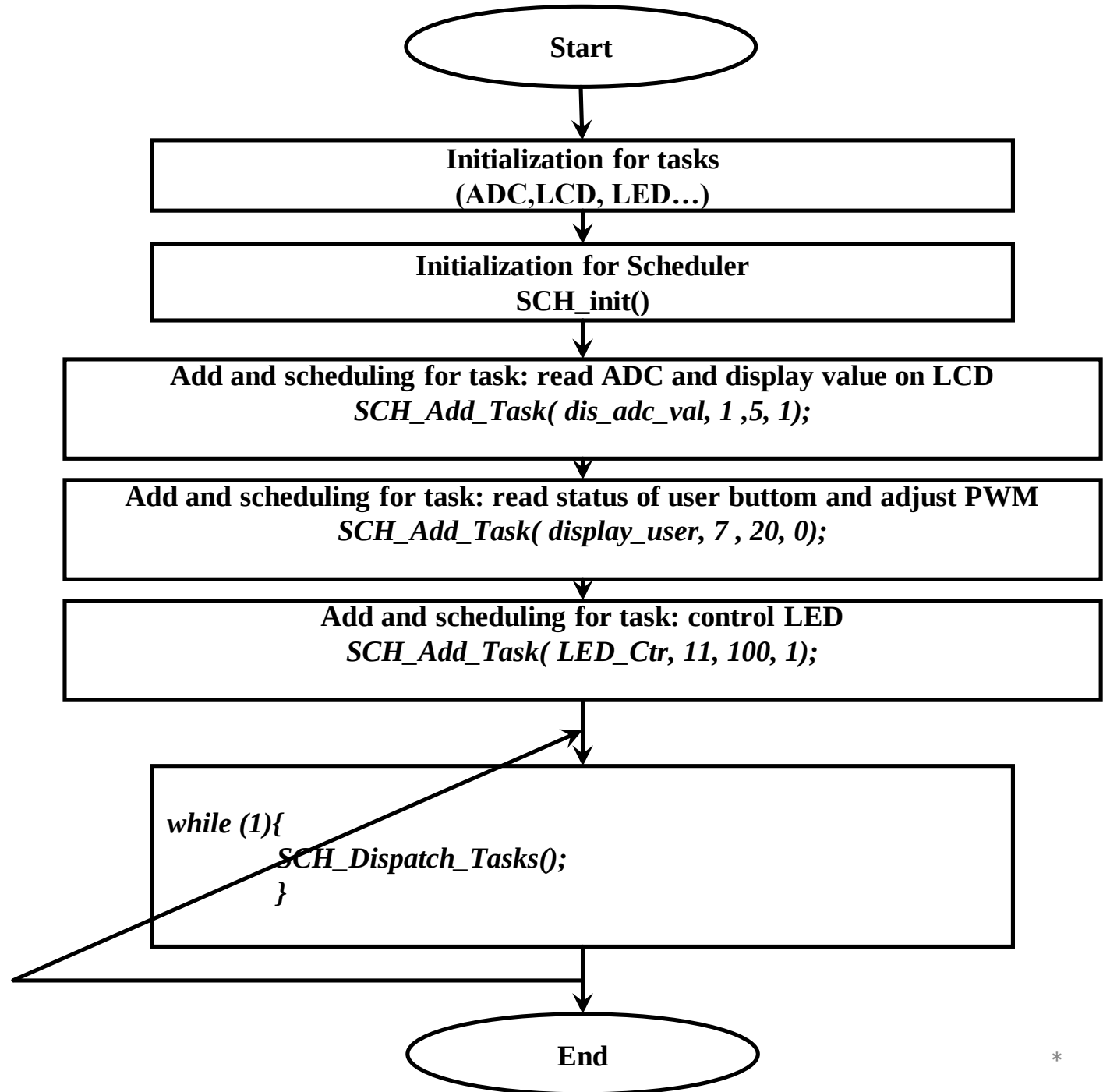


Tasks

- **void Read_ADC (void) {...}**
- **void LED_Ctr(void) {....}**
- **void PID_Ctr(void) {....}**
- **Void graph(void) {...}**
- **Void Send_data_PC(void) {...}**

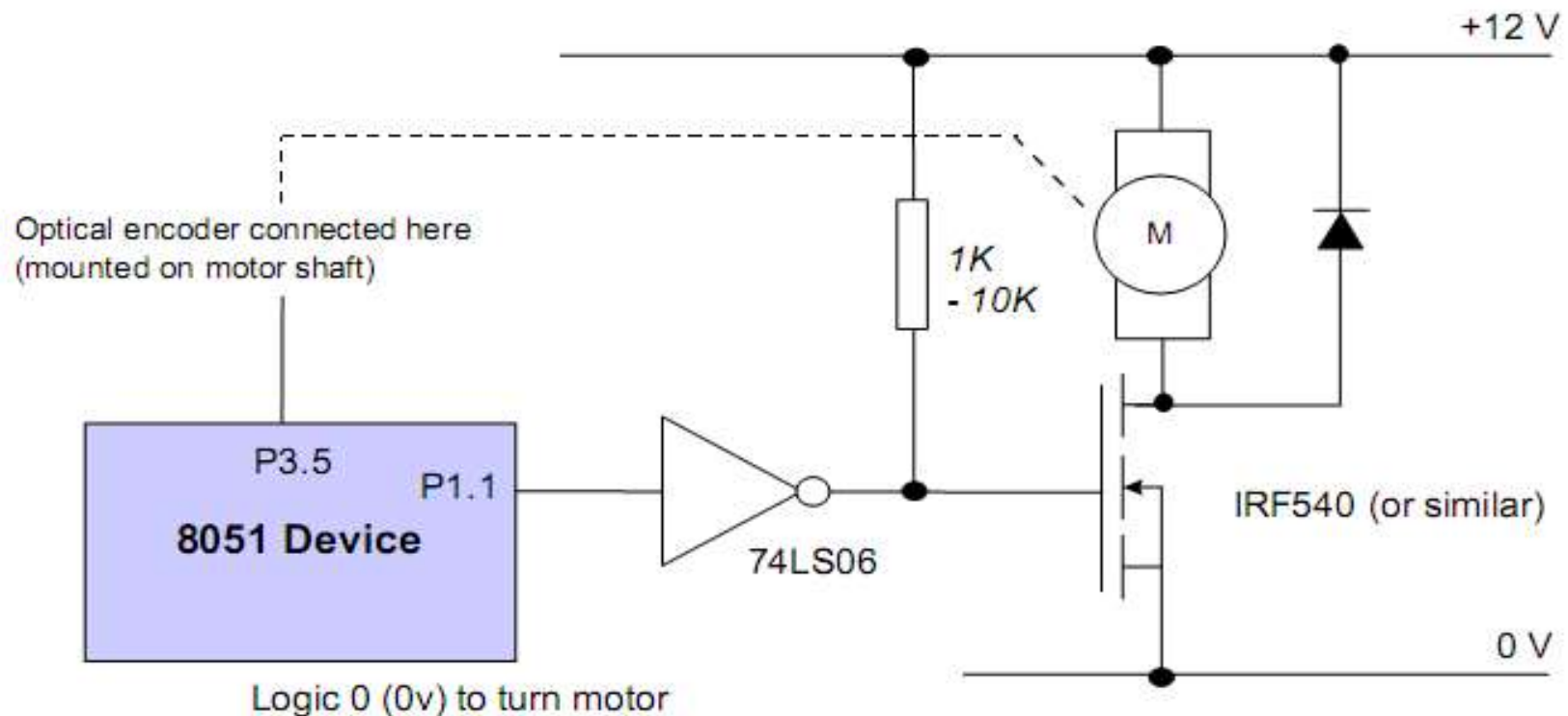
```
int main (void)
{
    STM32_init();
    ADC_init();
    LED_init();
    PWM_init();
    SCH_init();
    SCH_Add_Task(Read_ADC, 0, 2,0);
    SCH_Add_Task(PID_Ctr, 3, 5,1);
    SCH_Add_Task(LED_Ctr, 5,10,1);
    SCH_Add_Task(Graph, 9, 20, 1);
    SCH_Add_Task(Send_data_PC, 11,50,1);
    SCH_Start();
    While(1)
    {
        Dispatch();
    }
}
```

Main program



Example 4: PID in Motor Control using RTOS

- **Note:** this example uses a different, integer-based, PID implementation. As we discussed in 'Hardware resource implications', integer-based solutions impose a lower CPU load than floating-point equivalents.



```

void main(void)
{
    SCH_Init_T1(); /* Set up the scheduler */
    PID_MOTOR_Init();

    /* Set baud rate to 9600, using internal baud rate generator
    /* Generic 8051 version */
    PC_LINK_Init_Internal(9600);

    /* Add a 'pulse count poll' task */
    /* TIMING IS IN TICKS (1ms interval) */
    /* Every 5 milliseconds (200 times per second) */
    SCH_Add_Task(PID_MOTOR_Poll_Speed_Pulse, 1, 1);

    SCH_Add_Task(PID_MOTOR_Control_Motor, 300, 1000);

    /* Sending data to serial port */
    SCH_Add_Task(PC_LINK_Update, 3, 1);

    /* All tasks added: start running the scheduler */
    SCH_Start();

    while(1)
    {
        SCH_Dispatch_Tasks();
    }

    ...

#define PULSE_HIGH (0)
#define PULSE_LOW (1)

#define PID_PROPORTIONAL (5)
#define PID_INTEGRAL (50)
#define PID_DIFFERENTIAL (50)

```

```

void PID_MOTOR_Control_Motor(void)
{
    int Error, Control_new;

    Speed_measured_G = PID_MOTOR_Read_Current_Speed();
    Speed_required_G = PID_MOTOR_Get_Required_Speed();

    /* Difference between required and actual speed (0-255) */
    Error = Speed_required_G - Speed_measured_G;

    /* Proportional term */
    Control_new = Controller_output_G + (Error / PID_PROPORTIONAL);

    /* Integral term [SET TO 0 IF NOT REQUIRED] */
    if (PID_INTEGRAL)
    {
        Sum_G += Error;
        Control_new += (Sum_G / (1 + PID_INTEGRAL));
    }

    /* Differential term [SET TO 0 IF NOT REQUIRED] */
    if (PID_DIFFERENTIAL)
    {
        Control_new += (Error - Old_error_G) / (1 + PID_DIFFERENTIAL);

        /* Store error value */
        Old_error_G = Error;
    }

    /* Adjust to 8-bit range */
    if (Control_new > 255)
    {
        Control_new = 255;
        Sum_G -= Error; /* Windup protection */
    }

    if (Control_new < 0)
    {
        Control_new = 0;
        Sum_G -= Error; /* Windup protection */
    }

    /* Convert to required 8-bit format */
    Controller_output_G = (tByte) Control_new;

    /* Update the PWM setting */
    PID_MOTOR_Set_New_PWM_Output(Controller_output_G);
    ...
}

```